Acknowledgement

As I bring this project to completion, I am profoundly grateful for the support and encouragement I have received throughout this journey. From the initial brainstorming sessions to the final execution, I have been fortunate to receive the assistance of many remarkable individuals whose contributions have been invaluable.

First and foremost, I would like to express my heartfelt thanks to my lecturers for their unwavering dedication and inspiring guidance. Their expertise and encouragement have significantly enriched my knowledge and skills. I am especially grateful to Ms.Gayathri Karunarathna, whose exceptional mentorship has been pivotal in shaping my understanding and ensuring the success of this project.

I would also like to extend my appreciation to my classmates and peers, whose collaboration and constructive feedback have been an incredible source of motivation. Your insights and support have encouraged me to aim higher and embrace challenges with determination.

A special thank you goes to my family, whose steadfast belief in my abilities and constant encouragement have provided me with the strength and confidence to persevere. Your love and support have been the foundation of all my achievements.

Finally, I want to thank ESOFT Metro Campus for their assistance and for providing the resources needed to complete this assignment successfully. I also want to express my gratitude to everyone who has contributed to my academic and personal development.

Table of Contents

Plagiarism 3

Unit 35: System Analysis & Design 4

Please note that the resources listed are examples for you to use as a starting point in your report – the list is not definitive. 8

Activity 01 15

1.1 System 15

1.2 Computer Systems 15

1.3 System Analysis 16

1.4 Software Development Lifecycle. 17

1.5 Software Development Methodologies 20

1.5.1 Traditional Methodology 20

1.5.2 Agile Methodology 27

1.6 Compare the strengths and weaknesses of the two methodologies by referring to the proposed web system for "Fresh Grocers". 33

Activity 02 36

2.1 Feasibility Report 36

2.2 Analyzing each feasibility criteria identified and determining whether they are relevant to the problem statement 59

Activity 03 61

3.1 Strengths and Weaknesses of Traditional Methodology 61

3.2 Strengths and Weaknesses of Agile Methodology 64

3.3 Strengths and Weaknesses of Feasibility Study 67

3.4 Evaluation of Methodologies and Feasibility Study Effectiveness against the Problem Statement 70

3.5 Recommendations on the Most Suitable Methodology Based on Identified Strengths and Weaknesses 72

3.6 Proposed Solution for the Problem Statement 74

3.7 User Requirements of the System 76

3.7.1 Functional Requirement 76

3.7.2 Non-Functional Requirements 77

3.8 System Requirements 79

3.9 Reviewing the Requirement with fact finding techniques 81

3.10 Selecting a suitable methodology to further analyze and review the system requirements. 83
3.11 Implementing the selected methodology to further analyse and review the System Requirements 85
3.12 Assessing the effectiveness of the methodology used in providing a solution for a given business context. 87
Activity 04 89
4.1 User and System Requirements 89
4.2 System Designs 92
4.2.1 System Architecture 92
4.2.2 Data Flow Diagram (Level 0) 92
4.2.3 Data Flow Diagram (Level 01) 93
4.2.4 Use Case Diagram 94
4.2.5 User Interface Designs 95
4.2.6 Database Design 106
4.2.7 Programme Designs 107
4.3 Review the design and compare it against the user and system requirements. 116
4.4 Evaluating the methodology used to develop the system design and determine whether it was appropriate for the project. 119
4.5 Assessing the Effectiveness of the System Design 121
4.6 Justifying the choice of the analysis methodology used in the context of the business problem. 123
References 125

Table of Figures

Figure 1: Steps of SDLC 17
Figure 2: Planning phase of SDLC 18
Figure 3: Defining Requirements phase of SDLC 18
Figure 4: Designing phase of SDLC 18
Figure 5: Developing stage of SDLC 19
Figure 6: Testing stage of SDLC 19
Figure 7: Deployment and Maintenance phase of SDLC. 19
Figure 8: Waterfall Methodology (UseMotion.com) 26
Figure 9: System Architecture Design 92
Figure 10: Data Flow Diagram (Level 0) 92
Figure 11: Data Flow Diagram (Level 01) 93
Figure 12: Use Case Diagram 94
Figure 13: Wireframe: User Registration. 95
Figure 14: Prototype: Registration 96
Figure 15: Wireframe: Login 97
Figure 16: Prototype: Login 97
Figure 17: Wireframe: Customer Order Placing. 98
Figure 18: Prototype: Customer placing order 99
Figure 19: Wireframe: CSR placing order 100
Figure 20: Prototype: CSR placing order 101
Figure 21: Wireframe: Delivery Agent Dashboard. 102
Figure 22: Prototype: Delivery Agent dashboard 103
Figure 23: Wireframe: Customer Rating Delivery Agent. 104
Figure 24: Prototype: Customer rating Delivery Agent. 105
Figure 25: Database Design. 106
Figure 26: Flowchart: Registration. 107
Figure 27: Pseucocode: Registration. 108
Figure 28: Flowchart Customer Ordering. 109
Figure 29: Pseudocode: Customer Ordering 110
Figure 30: Flowchart: CSR Ordering 110
Figure 31: Pseudocode: CSR Ordering 111
Figure 32: Flowchart: Delivery Management. 112
Figure 33: Pseudocode: Delivery Management. 113
Figure 34: Flowchart: Delivery Agent Update Location. 114
Figure 35: Pseudocode: Delivery Agent Update Location. 115

Activity 01

1.1 System

A system is a collection of interconnected components that function together to achieve a specific purpose. It can be found in various domains, such as:

- **Mechanical Systems** – Examples include car engines, turbines, and conveyor belts, where physical components interact to perform a task.
- **Biological Systems** – The human body, ecosystems, and digestive systems illustrate how living organisms and processes work harmoniously.
- **Digital Systems** – Computers, software applications, and network infrastructures enable data processing, automation, and communication.
- **Economic Systems** – Market structures, supply chains, and financial networks facilitate commerce and transactions.
- **Social Systems** – Communities, legal frameworks, and governance models regulate societal interactions and behaviors.

Each system has inputs, processes, and outputs, functioning in an organized manner to maintain stability or adapt to change. The concept of systems is fundamental to problem-solving, innovation, and understanding complex interactions in various fields.

1.2 Computer Systems

A computer system is a combination of hardware and software that work collectively to process data and perform tasks. It typically consists of:

- Hardware - physical elements like the CPU (Central Processing Unit), RAM (memory), storage devices (HDD, SSD), input devices (keyboard, mouse), and output devices (monitor, printer).

- Software- Programs that run on the hardware, include operating systems (Windows, Linux, macOS) and apps (browsers, word processors, games).
- Network & Connectivity- Components that facilitate connection between systems, such as Wi-Fi, Ethernet, routers, and internet services.
- Data & Storage - Systems for managing and storing information, including databases and cloud storage.
- Users - People or automated processes engaging with the system for various functions.

Computer systems span from modest personal computers to huge networked systems utilized in corporations and industries. They play a key part in modern life, supporting communication, automation, computation, and entertainment.

1.3 System Analysis

System analysis is the process of evaluating a system to understand its structure, functioning, and relationships. It assists in recognizing problems, establishing needs, and designing solutions for better efficiency.

Importance of system analysis:

1. **Problem identification** - Helps recognize inefficiencies and areas for improvement.
2. **Requirement definition** -Describes user needs and system functionalities for better design.
3. **Improved efficiency** - Optimizes workflows, resource use, and system performance.
4. **Enhanced decision-making** -Provides insights for educated choices in system development.
5. **Minimization of hazards** -Reduces errors, redundancies, and operational failures.
6. **Cost savings** -Streamlines operations and reduces unnecessary expenses.
7. **Scalability and future growth** - Ensures system can adapt to future changes.

System analysis is commonly utilized in software development, business process optimization, and engineering systems.

1.4 Software Development Lifecycle.

Software development life cycle (SDLC) is a structured procedure that is used to design, construct, and test good-quality software. It is a technique that specifies the whole practice of software development step-by-step. The goal of the SDLC life cycle model is to generate high-quality, maintainable software that meets the user's needs. SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model may do its duty efficiently to offer the product at a low cost within a defined time frame that fits customer needs. In this article we shall explore Software Development Life Cycle (SDLC) in complete. (Anon., 2024)

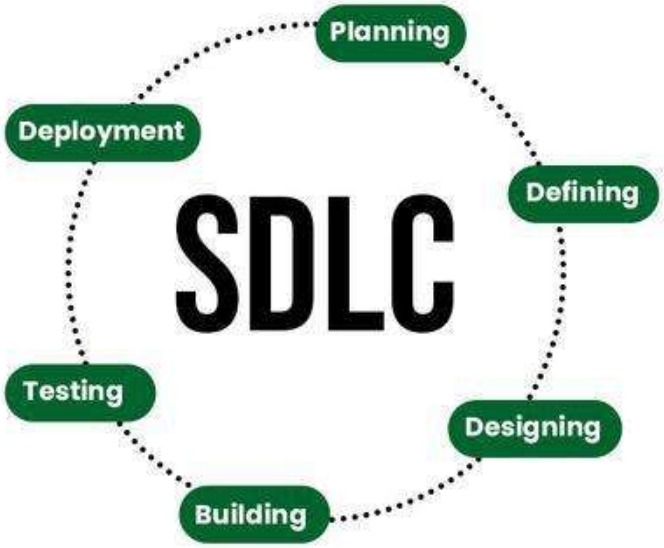


Figure 1: Steps of SDLC

There are 06 steps in SDLC. They are;

01. Planning – Requirement analysis is done by the developers of the organization. This results of this analysis constitute the building elements of the project.

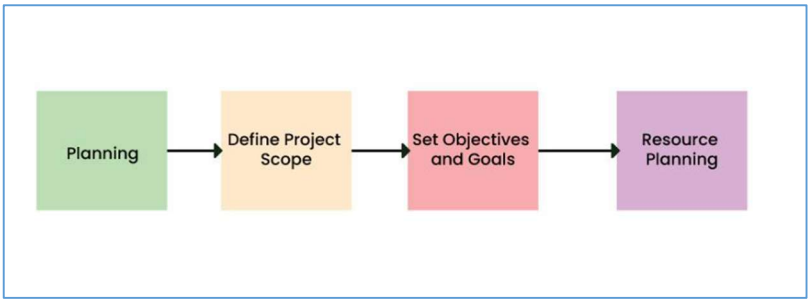


Figure 2: Planning phase of SDLC

02. Defining criteria - All the criteria required for the software are described. These standards are then approved by users and stakeholders. This is done by leveraging Software Requirement Specification document.

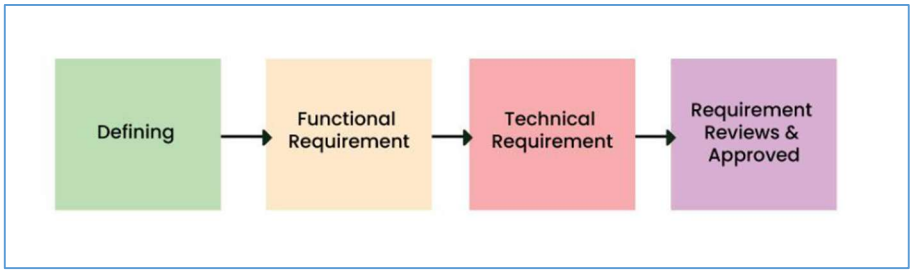


Figure 3: Defining Requirements phase of SDLC

03. Software designers come out with a best architecture design by using the SRS document.

    Multiple designs are offered in the Design Document Specification (DDS). These designs are verified by analysts and stakeholders, and the most suited one is picked.

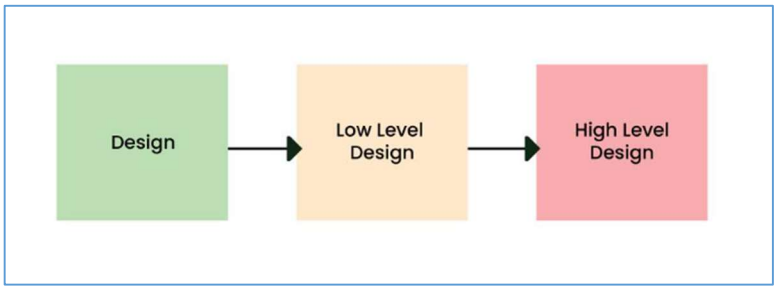


Figure 4: Designing phase of SDLC

04. Developing - The main development of the product starts here. Developers employ a specific programming code as per the DDS design. Conventional coding instruments like compilers, interpreters, debuggers, etc. are also put into use at this level.

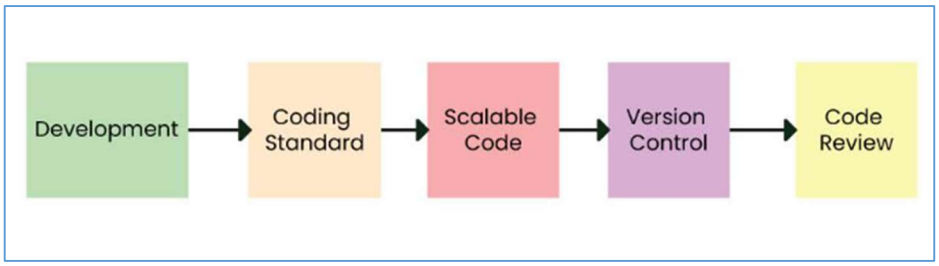


Figure 5: Developing stage of SDLC

05. Testing and Integration- After the creation of the product, testing of the software is important to assure its seamless execution. At this stage, all the possible problems are tracked, repaired, and retested.

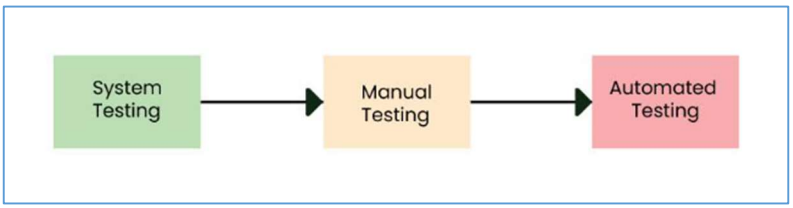


Figure 6: Testing stage of SDLC

06. Deployment and Maintenance- After rigorous testing, the definitive product is launched in phases and tested in a genuine industrial environment. After extracting beneficial comments, the company releases it as it is or with improvements. (Anon., 2024)

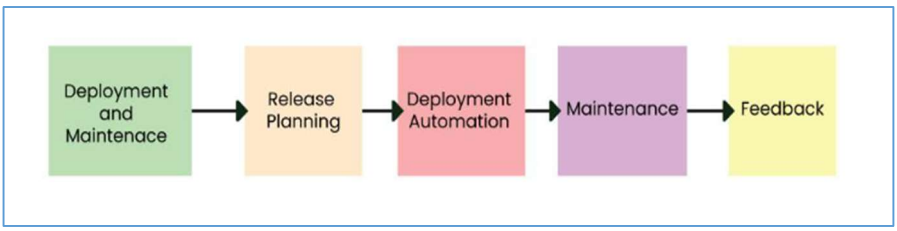


Figure 7: Deployment and Maintenance phase of SDLC.

1.5 Software Development Methodologies.

Software development techniques are systematic procedures used to design, build, and maintain software efficiently. These approaches help teams plan tasks, manage resources, and maintain quality throughout the development process.

There are two primary types:

Traditional - Also known as the Waterfall model, this follows a linear and sequential method where each phase (planning, design, implementation, testing, and deployment) is completed before advancing to the next. It works effectively for projects with defined criteria but can be strict when adjustments are needed.

Agile - Focuses on iterative development, where software is produced in tiny, incremental phases called sprints. Agile fosters flexibility, collaboration, and continual feedback, allowing teams to adjust swiftly to changing requirements.

Each technique has its strengths depending on the project type and aims.

1.5.1 Traditional Methodology

Traditional methodology, also referred to as the Waterfall model, is a systematic approach to project management that follows a linear and sequential procedure. It is extensively employed in businesses where requirements are well-defined and unlikely to alter. The technique comprises of numerous discrete phases:

Initiation- This phase specifies the project's objective, scope, and feasibility. Stakeholders determine objectives, risks, and restrictions, ensuring the project matches with company goals. A feasibility study may be done to examine viability.

Planning– Detailed project plans are created, including schedules, resource allocation, budgeting, and risk management measures. Teams define milestones and deliverables, creating a clear path for implementation.

Execution- The real development or implementation occurs in this phase. Teams follow the predefined strategy, executing tasks according to requirements. Coordination among departments is vital to achieve seamless progress.

Monitoring and Controlling- Progress is tracked against the initial plan, assuring adherence to schedules and quality standards. Any aberrations are addressed through corrective actions, and risks are controlled to prevent interruptions.

Closure - The project is finalized, reviewed, and given to stakeholders. Documentation is completed, lessons learned are recorded, and post-project evaluations help enhance future operations. (Reaiche & Papavasiliou, n.d.)

Types of Waterfall Methodology

- **Waterfall model -** A linear approach where each phase must be finished before going to the next. Used in sectors needing tight documentation and regulatory compliance.
- **Critical path method (CPM) -** Focuses on identifying the longest chain of dependent jobs to maximize scheduling. Commonly used in building and engineering projects.
- **Critical chain project management (CCPM) -** Prioritizes resource allocation and buffers to prevent delays. Applied in manufacturing and large-scale industrial projects.
- **V-Model** - An augmentation of the Waterfall model that includes testing phases with development stages. Used in safety-critical areas including healthcare and aerospace.
- **Spiral model -** Combines iterative development with risk assessment, making it suited for complicated projects with dynamic requirements.

Traditional techniques are best suited for projects with constant requirements and known outputs, ensuring disciplined execution and accountability

Strengths of Waterfall Methodology with Empirical and Practical Evidences.

Simple and Easy to Understand

Description: Waterfall's linear and sequential structure makes it straightforward to grasp and manage, especially for teams new to formal development processes.

Evidence: Sommerville (2011) states that Waterfall's well defined phases allow teams and stakeholders understand progress readily and allocate responsibilities clearly. (Sommerville, 2011)

Well-Defined Documentation

Description: Each phase creates particular deliverables and documentation, facilitating knowledge transfer and project tracking.

Evidence: Pressman (2014) underlines Waterfall's strength in creating rich documentation that supports maintenance and future upgrades. (Pressman, 2014)

Structured Approach with Clear Milestones

Description: Waterfall's phase-driven methodology provides distinct milestones and deliverables that make project management and progress reporting straightforward.

Evidence: A research by Benington (1983) revealed that Waterfall's organized approach benefits project managers in scheduling and resource allocation. (Benington, 1983)

Good for Projects with Stable Requirements

Description: When needs are well understood and unlikely to change, Waterfall is efficient and effective.

Evidence: Larman and Basili (2003) discovered Waterfall works well in situations such as government and defense programs where criteria are established and tight compliance is important. (Larman & Basili , 2003)

Ease of Management and Control

Description: The sequential phases with clearly defined start and end points make it easier for project managers to control the development process.

Evidence: Royce (1970) stated that the Waterfall model provides managers with a foundation for disciplined project control. (Royce, 1970)

Facilitates Early System Design

Description: Waterfall needs thorough system design early in the lifecycle, which might detect architectural or feasibility difficulties before coding begins.

Evidence: Boehm (1988) showed that upfront design helps ensure the system architecture is robust, which is crucial in specific industries like aerospace and defense. (Boehm, 1988)

Highly Disciplined Process

Description: Waterfall promotes a disciplined, methodical approach to software development, which aids regulatory or safety-critical projects.

Evidence: Pfleeger and Atlee (2006) showed that Waterfall's rigor makes it acceptable for areas such as medical, aviation, and finance, where compliance and traceability are necessary. (Pfleeger & Atlee, 2006)

Weaknesses of Waterfall Methodology with Empirical and Practical Evidences

Inflexibility to Changes

Description: Waterfall involves a linear and sequential procedure, making it difficult to accommodate modifications once a step is done. Requirements must be thoroughly understood upfront, yet in real projects, requirements often evolve.

Evidence: Boehm (1988) noted that early design decisions constrain the later phases, and after a stage is done, going back to make changes is costly and time-consuming, sometimes resulting to project failure or overruns. (Boehm, 1988)

Late Testing Phase

Description: Testing occurs only after the implementation phase, which delays the detection of faults. This often results in higher costs and time delays when faults are recognized late.

Evidence: Larman and Basili (2003) discovered through case studies that the Waterfall model's late testing phase leads to greater defect identification costs relative to iterative alternatives. (Larman & Basili , 2003)

Poor Handling of Requirement Uncertainty

Description:

Waterfall presupposes steady and well-defined needs, which is rarely the case in dynamic business contexts. This results in mismatched products and user displeasure.

Evidence: In a research by Royce (1970), the original Waterfall idea was defined as problematic due to the unrealistic assumption that needs are fixed, which was reinforced by several project failures. (Royce, 1970)

Lack of Customer Involvement

Description: Customers are only involved at the beginning and finish, decreasing their capacity to change the product based on iterative input, which may lead to a final product that doesn't fulfill expectations.

Evidence: A study by Boehm and Turner (2004) demonstrated that projects with limited client interaction under Waterfall generally produced misaligned outcomes compared to Agile alternatives with continuous feedback. (Boehm & Turner, 2004)

Risk Management is Poor

Description: Waterfall does not involve continual risk assessment and mitigation throughout the project, rendering it prone to unforeseen problems.

Evidence: Boehm (1988) proposed the Spiral Model partly due to Waterfall's failure to handle risks proactively, with empirical evidence showing projects failing when problems were detected late. (Boehm, 1988)

Long Delivery Times

Description: Because of its sequential structure, Waterfall programs give the complete result only at the end, delaying value realization and feedback.

Evidence: A survey by VersionOne (2017) indicated that teams adopting Waterfall had higher cycle times and slower delivery compared to Agile teams, harming business agility. (VersionOne, 2017)

Difficulty in Handling Complex and Large Projects

Description: Waterfall is less appropriate for complicated projects where needs evolve and where incremental development is helpful.

Evidence: Boehm and Turner (2004) identified several large-scale project failures using Waterfall, proposing Agile and hybrid approaches for better adaptation and complexity management. (Boehm & Turner, 2004)

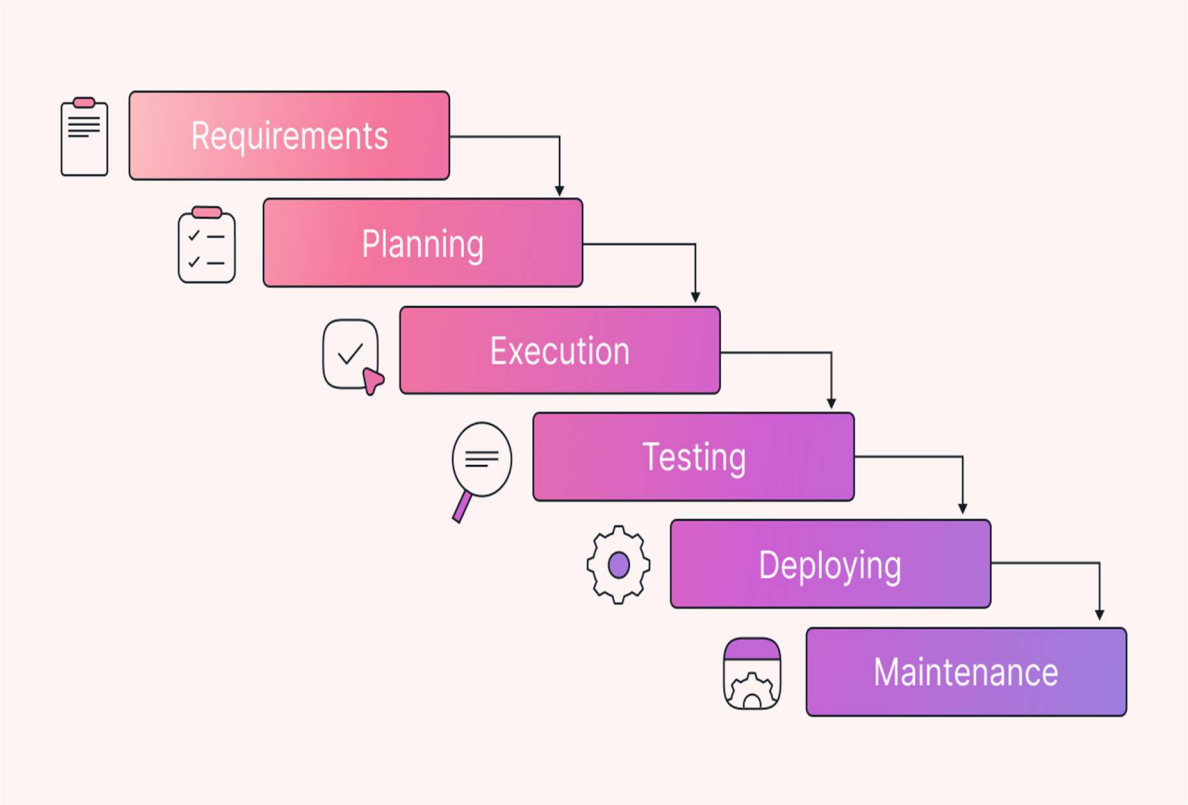


Figure 8: Waterfall Methodology (UseMotion.com)

1.5.2 Agile Methodology

Agile methodology is a flexible and iterative approach to software development and project management that emphasizes on continuous improvement, customer input, and rapid delivery of functional product. Unlike traditional methods like Waterfall, Agile does not follow a straight path; instead, it separates the work into smaller chunks called iterations or sprints, often lasting 1 to 4 weeks. Agile fosters strong collaboration between cross-functional teams and stakeholders, adaptation to changes, and constant improvement of both product and process. The essential values of Agile are founded on the Agile Manifesto, which emphasizes persons and interactions over processes and technologies, working software over detailed documentation, customer collaboration over contract negotiation, and adapting to change over following a plan.

Phases of Agile Methodology

- **Concept or requirements gathering:** when stakeholders and the development team collaborate to identify business needs and define user stories—simple, brief descriptions of a product from the perspective of the user.
- **Iteration or sprint planning:** This is where the team pick which user stories from the product backlog will be worked on during the sprint. These stories are split down into smaller jobs, and team members estimate the time and effort required for each work.

- **Design phase:** where the team constructs a minimal design or architecture to support the selected user stories. This may contain wireframes, system diagrams, or interface prototypes, but the focus remains on basic and effective design to start creating rapidly.
- **Development phase:** During which the team writes the actual code to implement the features. Agile emphasizes approaches such as pair programming and continuous integration, helping teams to spot errors early and maintain excellent code quality.
- **Testing phase:** where they conduct unit tests, integration tests, and user acceptance testing to ensure the software functions as intended. Bugs are identified, resolved, and retested.
- **Release or deployment phase:** Team comprises providing the working software to stakeholders or deploying it to a staging or production environment.
- **Sprint review:** The team demonstrates the completed features to the stakeholders, receives input, and checks whether the sprint goals were accomplished.
- **Retrospective phase:** lets the team to reflect on their performance during the sprint. They examine what went well, what challenges they had, and how they may better in the upcoming sprint. This continual learning loop helps the team become more efficient and productive with each iteration.

Types of Agile Methodologies

- **Scrum:** One of the most extensively used frameworks. It organizes work into time-boxed sprints and defines particular roles such as Product Owner, Scrum Master, and Development Team. Scrum stresses daily stand-up meetings, sprint reviews, and retrospectives to maintain clear communication and continual progress.
- **Kanban:** Another famous Agile method that focuses on visualizing the workflow. It uses a board with columns like "To Do," "In Progress," and "Done" to handle tasks. Kanban strives to restrict work in progress and enhance flow efficiency, making it perfect for teams that want flexibility and continuous delivery without fixed-length sprints.
- **Extreme Programming (XP):** An Agile methodology that stresses technical quality and practices such as test-driven development (TDD), pair programming, continuous

integration, and frequent releases. XP is best suited for projects that need high-quality code and when requirements regularly change.

- **Lean:** Inspired by lean manufacturing concepts, attempts to minimize waste, enhance efficiency, and deliver value rapidly. It emphasizes a customer-focused approach, constant learning, and making decisions as late as possible to provide flexibility.
- **Crystal:** A collection of Agile techniques that change based on the size and criticality of the project. It encourages simplicity, collaboration, and regular delivery, and thinks that the finest processes come from the team itself rather than inflexible frameworks.
- **Feature-Driven Development (FDD):** Focuses on building and designing features as the major unit of progress. It follows a disciplined methodology that begins with modelling the domain and then creating features in short, planned iterations. FDD is typically utilized in large-scale company projects that demand a more regulated Agile approach.

Strengths of Agile Methodology with Empirical and Practical Evidences

High Flexibility and Adaptability to Change

Description: Agile accepts new requirements even late in development, letting the product to grow according to user needs and market trends.

Evidence: VersionOne's 14th State of Agile Report (2020) indicated that 70% of Agile adopters named ability to manage changing priorities as a top benefit. (VersionOne, 2020)

Customer-Centric Development

Description: Agile supports ongoing customer involvement through feedback loops, resulting in a product that better matches with consumer expectations.

Evidence: Highsmith (2002) underlined Agile's iterative customer feedback process leads to greater product satisfaction and business value. (Highsmith, 2002)

Faster Time to Market

Description: Agile creates working software in short iterations (sprints), enabling quicker release cycles and early value delivery.

Evidence: A research by Poppendieck & Poppendieck (2003) revealed that Agile's incremental delivery methodology lowers the time between idea and product release. (Poppendieck & Poppendieck, 2003)

Improved Team Collaboration and Communication

Description: Agile stresses intimate collaboration through daily stand-ups, sprint reviews, and retrospectives, leading to better coordination and morale.

Evidence: Cockburn and Highsmith (2001) noted that Agile teams tend to be more communicative and responsive, enhancing productivity and satisfaction. (Highsmith & Cockburn, 2001)

Higher Product Quality

Description: Agile combines continuous testing, integration, and refactoring, which helps find errors early and improve code quality.

Evidence: A study by Dingsøyr, Nerur, Balijipally and Moe (2012) indicated that Agile approaches like Test-Driven Development (TDD) and Continuous Integration increase software quality. (Dingsoyr, et al., 2012)

Increased Stakeholder Engagement

Description: Stakeholders are actively engaged throughout the development process, boosting transparency and alignment with company goals.

Evidence: Beck, Beedle and Bennekum (2001), in the Agile Manifesto, highlighted customer cooperation as a core value, which has demonstrated to boost stakeholder satisfaction in practice. (Beck, et al., 2001)

Continuous Improvement and Learning

Description: Agile pushes teams to reflect constantly on their performance and incorporate improvements through sprint retrospectives.

Evidence: Schwaber and Sutherland (2017) noted that Scrum's inspect-and-adapt cycles allow teams to continually change and enhance their procedures. (Schwaber & Sutherland , 2017)

Weakness of Agile Methodology with Empirical and Practical Evidences

Lack of Predictability

Description: Due to its fluid and adaptive nature, Agile generally lacks clear schedules, scope, and budgets, making it tougher for stakeholders to predict outcomes.

Evidence: Boehm and Turner (2004) highlighted that Agile projects can suffer from unpredictability in scope and expense, which is an issue in contract-driven or regulated organizations. (Boehm & Turner, 2004)

Requires High Customer Involvement

Description: Agile relies significantly on frequent customer input. If the customer is inaccessible or unengaged, the project may lack direction and lead to misaligned deliveries.

Evidence: Ambler (2002) stated that one of the main hazards in Agile is the assumption that clients will be readily available throughout development. (Ambler, 2002)

Difficult to Scale for Large Projects

Description: Agile approaches, particularly Scrum, perform well with small teams, but coordination and communication become hard as team sizes and project complexity expand.

Evidence: Dingsøyr, Moe, Faegri and Seim (2012) observed that large-scale Agile adoption required extra layers (like SAFe or LeSS), adding complexity and sometimes weakening agility.

Lack of Comprehensive Documentation

Description: Agile prioritizes functioning software above thorough documentation, which can be challenging for maintenance, audits, or knowledge transfer after project completion.

Evidence: Sommerville (2011) pointed out that Agile's lightweight documentation often leads to knowledge loss, especially in long-term or heavily regulated projects. (Sommerville, 2011)

Requires Experienced and Cross-Functional Teams

Description: Agile presupposes a mature, self-organizing team with technical and collaborative expertise. Inexperienced teams may struggle to make decisions or manage themselves efficiently.

Evidence: Highsmith (2002) stated that Agile teams demand a high level of discipline and experience, and without this, productivity and quality might suffer. (Highsmith, 2002)

Potential for Scope Creep

Description: Because Agile embraces modifications at any level, projects can suffer from uncontrolled scope expansion, hurting time and budget.

Evidence: VersionOne's State of Agile Report (2020) placed managing changing priorities and scope creep among the top issues experienced by Agile teams. (VersionOne, 2020)

Not Ideal for All Types of Projects

Description: Agile is not ideal for projects with fixed contracts, stringent compliance requirements, or when a comprehensive upfront design is necessary.

Evidence: Boehm (2002) explained that Agile may not fit with high-assurance systems (e.g., aerospace, medical) where thorough documentation and verification are necessary from the outset. (Boehm, 2002)

1.6 Compare the strengths and weaknesses of the two methodologies by referring to the proposed web system for "Fresh Grocers".

Project Success Rates

General Comparison: Agile techniques tend to offer much greater project success rates due of their iterative nature, transparency, and ongoing stakeholder involvement. Agile supports early development of core features and continuous testing, enabling teams to release working software at frequent intervals. This means problems are recognized early, and adjustments are made routinely, leading to improved alignment with corporate goals. The Standish Group's CHAOS Report (2015) indicated that 39% of Agile projects succeeded (on schedule, on budget, with required features), while only 11% of Waterfall initiatives accomplished the same. Waterfall's linear approach prevents visibility into the finished product until very late, increasing the likelihood of misalignment with user needs.

In the Context of Fresh Grocers: Fresh Grocers is constructing a feature-rich, interactive platform where failing to meet consumer expectations such as real-time delivery tracking or order confirmation SMS can result in lost customers. Using Agile, Fresh Grocers can first deliver a minimum viable product (MVP) with critical functionality like client registration,

order placing, and SMS notifications. Feedback from clients and delivery people can then be utilized to influence future improvement. If Agile is followed, the system grows naturally with feedback, enhancing the likelihood of a successful launch. In contrast, with Waterfall, if delivery personnel registration or the SMS system fails during final testing, correcting it may delay the entire deployment and undermine customer trust.

Flexibility and Adaptability

General Comparison: Agile is built for change. It embraces developing requirements, even late in development. Each sprint allows teams to reassess and re-prioritize the backlog based on current conditions or input. This adaptability is ideal for fast-moving environments. Waterfall, in contrast, needs tight adherence to initial ideas. Once the design process is complete, adjustments are difficult and costly, frequently requiring major rework. This rigidity can limit innovation or adaptation to market conditions.

In the Context of Fresh Grocers: Fresh Grocers may meet shifting client habits post-launch such as growing demand for contactless delivery, live order tracking, or payment integrations like QR code scanning. An Agile framework allows developers to respond to such demands promptly, introducing additional features in future sprints. This maintains the system relevant and competitive. If Waterfall is employed, any such demand changes would require a formal change process, perhaps pushing them to a second-phase release months later by which time Fresh Grocers may lose consumers to more adaptive competitors.

Requirements Management

General Comparison: Waterfall presupposes that all project needs can be known and recorded at the start. This is viable in well-defined, unchanging contexts. However, in most real-world initiatives, requirements vary owing to stakeholder feedback or business shifts. Agile fights this by focusing on incremental requirement collecting through user stories, backlog grooming, and sprint planning. Continuous involvement ensures the solution always corresponds with genuine user demands.

In the Context of Fresh Grocers: The change from physical grocery delivery to a digital ordering platform presents new obstacles and expectations. For instance, Fresh Grocers might not initially grasp the need to let people re-order commonly purchased items or the

necessity of multilingual interfaces. With Agile, such new needs can be added iteratively without affecting the primary project. Delivery personnel or customer reps can raise concerns like difficulty in manual location updates which can be addressed in following sprints. In contrast, Waterfall would fix the scope early. If Fresh Grocers overlooked any of these needs in the initial planning, fixing them would involve beginning a new project or postponing deployment.

Quality and Testing

General Comparison: Agile embeds testing into each development cycle. Developers and testers cooperate from the beginning, employing approaches like continuous integration, unit testing, and sprint-end reviews. This strategy ensures that faults are discovered and remedied early, maintaining good software quality throughout the project. Waterfall, however, places testing at the end of the lifecycle. Bugs uncovered during this phase may affect numerous areas of the system and necessitate redesigning previously built components, which is time-consuming and expensive.

In the Context of Fresh Grocers: Fresh Grocers' system supports real-time and multi-party interactions like syncing delivery person location with client orders and sending automated SMS warnings. These technologies must perform flawlessly to preserve confidence and convenience. Agile's iterative testing strategy ensures that as each feature is built say, customer registration or location matching it's tested for both function and integration. Developers may immediately resolve issues, reducing post-deployment bugs. Waterfall's delayed testing could find issues such as improper agent assignment logic or unsuccessful SMS dispatching only at the final stage, causing delivery disruptions and customer complaints at launch.

Team Collaboration and Communication

General Comparison: Agile fosters strong team collaboration through ceremonies including daily stand-ups, sprint planning, retrospectives, and collaborative documentation. All stakeholders including developers, testers, product owners, and business users, participate throughout the process. This fosters common understanding, faster decision-making, and quicker response to issues. Waterfall, on the other hand, often splits teams into segregated

functions. Communication between business users and technical teams may deteriorate after early planning, resulting to misunderstandings and misaligned outputs.

In the Context of Fresh Grocers: Successful implementation at Fresh Grocers involves good coordination between development teams and frontline staff such as delivery agents and customer support reps. Agile allows these operational users to submit feedback continuously such as demanding a speedier manual order entry interface or highlighting ambiguous location inputs. This direct involvement results in a system adapted to real-world needs. With Waterfall, communication may occur just during the initial requirement gathering phase. If operational staff identify inefficiencies later, they must wait for the next system change, significantly hurting adoption and satisfaction.

Customer Satisfaction

General Comparison: Agile is customer-centric, with a strong focus on delivering working products early and frequently. Customers are involved at every step, reviewing product increments, expressing input, and influencing the product direction. This leads to increased customer satisfaction as the final product is more aligned with expectations. Waterfall lacks constant client connection, so if the product doesn't match user needs at delivery, there's minimal possibility for fast improvement.

In the Context of Fresh Grocers: Customer happiness is crucial to Fresh Grocers' success in the e-commerce sector. Agile enables Fresh Grocers to introduce features like order confirmations, delivery time estimates, and driver ratings early, then update them with real feedback. For example, customers can recommend a better approach to pick preferred delivery times, and this can be incorporated rapidly in a future sprint. Waterfall's "big bang" launch method may provide a system that seems spectacular but misses little elements that dramatically influence user experience. If feedback identifies serious usability faults, it may take months to resolve them by which time customers may shift to more responsive competitors.

Activity 02

2.1 Feasibility Report

FEASIBILITY REPORT

Prepared by:
Isma Haseemdeen



Web-based Platform- Fresh Grocers

➢ Background:

Fresh Grocers is a long-standing grocery delivery service in Sri Lanka, operating successfully for over fifteen years. The company has built a strong customer base by offering reliable and timely delivery of groceries to households in urban and suburban areas. Over the years, Fresh Grocers has gained a reputation for quality service, delivering a wide range of fresh produce and essential goods directly to customers' homes, contributing to its recognition as a trusted name in local grocery delivery.

The business model of Fresh Grocers traditionally relies on manual order processing and phone-based communication. Customers place orders via telephone, which are then entered and managed by customer service representatives. Delivery agents are assigned to orders based on their availability and proximity to the delivery location, often determined manually. Although effective in the past, this system lacks the flexibility, speed, and user experience demanded by today's digitally connected consumers.

To remain competitive and meet growing customer expectations, Fresh Grocers has decided to implement a modern web-based platform. This new system will enable customers to register and place orders online, receive automated SMS and email updates, select nearby delivery agents, and rate their delivery experience. It will also empower delivery agents to manage their availability and location within the system, and allow customer service staff to assist customers who are unable to place orders themselves. By embracing digital transformation, Fresh Grocers aims to streamline operations, improve customer satisfaction, and strengthen its position in the rapidly evolving online grocery market in Sri Lanka.

➢ Problem Definition

Currently, Fresh Grocers operates through a manual, telephone-based ordering system. Customers call a customer service representative to place their grocery orders, which are then recorded manually. Delivery agents are contacted individually and assigned based on their availability and approximate location, usually estimated without any real-time tracking or location data. All communications, including order confirmations and delivery updates, are handled manually through phone calls or SMS sent by staff. This process, though functional, is time-consuming and heavily dependent on human coordination. The existing system presents several critical problems. First, it lacks automation and digital tracking, which leads to delays and inefficiencies in order processing and delivery assignment. Without real-time location data, assigning the nearest delivery agent is often inaccurate, resulting in longer delivery times and higher fuel costs. Second, there is no platform for customers to view available products, track orders, or provide feedback, which limits customer engagement and satisfaction. Additionally, the current system does not support customer registration or digital authentication, making it difficult to personalize service or maintain customer order history. Furthermore, the manual handling of orders and delivery coordination increases the risk of human error, such as missed orders, incorrect deliveries, or delayed communication. The absence of automated SMS/email notifications and lack of integration between customers, delivery agents, and the customer service team reduce overall service efficiency. In an increasingly competitive market, continuing to rely on outdated processes may lead to customer attrition and lost market share. These challenges highlight the urgent need for a web-based system that can streamline operations, reduce errors, and deliver a modern, user-friendly experience.

➢ Problem Solution

To overcome the limitations of the current manual system, Fresh Grocers proposes the development and implementation of a web-based grocery ordering and delivery management platform. This system will introduce a centralized, automated solution that enhances efficiency, accuracy, and customer satisfaction. The platform will streamline the entire process; from customer registration and order placement to delivery tracking and feedback collection, by integrating modern digital technologies. The new system will allow both customers and delivery personnel to register through the website, upon which they will receive a unique username and password via email for secure login. Customers will be able to browse products, place orders online, and specify their delivery location. Upon order confirmation, they will receive an automated SMS containing the delivery agent's contact details and estimated delivery time. This automation reduces the need for manual communication and improves the speed and clarity of order handling. Additionally, the system will feature a location-based assignment module that identifies and displays the delivery agents closest to the customer's location, allowing either the customer or the customer service representative to select the most suitable agent. Delivery agents can manually update their current location through the system, while customer service staff can assist customers who prefer to place orders by phone, ensuring inclusivity. The platform will also include a customer feedback feature, enabling users to rate delivery agents after each order, which helps maintain service quality. Overall, this solution addresses the inefficiencies of the old system by digitizing key operations and delivering a faster, more transparent, and customer-friendly service experience.

➢ Scope of the System

The proposed web-based system for Fresh Grocers will focus on automating key aspects of the grocery ordering and delivery process to enhance operational efficiency and improve customer experience. The system will serve multiple user roles, including customers, delivery personnel, and customer service representatives, offering a user-friendly interface tailored to each role's needs.

The core functions that will be automated include customer and delivery agent registration, order placement, delivery assignment, location tracking, and notification services. Upon registration, users will automatically receive their login credentials via email. Customers will be able to place orders online, eliminating the need for manual order entry. Once an order is confirmed, the system will automatically send an SMS to the customer with the delivery agent's contact information and estimated delivery time.

Additionally, the system will automate the process of identifying and displaying nearby delivery agents based on the customer's location, improving the speed and accuracy of agent assignment. Delivery agents can update their location manually, and customer service representatives can input customer and location data when handling phone-based orders. The system will also automatically record customer feedback and delivery ratings after each completed order. By automating these processes, the platform minimizes human error, reduces operational delays, and ensures a more consistent and efficient service experience for all users.

➢ Objective of the System

- **Online Registration for Customers and Delivery Personnel:**
  Allow both customers and delivery agents to register on the web platform. After registration, the system will automatically send an email containing their unique username and password for secure access.

- **Web-Based Grocery Ordering:**
  Enable customers to conveniently place grocery orders through the website, streamlining the ordering process without needing phone calls or manual entry.

- **Automated SMS Notifications:**
  After an order is confirmed, the system will automatically send an SMS to the customer with the delivery agent's contact information and the estimated delivery time to keep customers informed.

- **Delivery Personnel Rating System:**
  Provide customers the ability to rate delivery agents based on their delivery experience, allowing the company to monitor service quality and improve customer satisfaction.

- **Order Entry by Customer Service Representatives:**
  Allow customer service staff to manually enter orders on behalf of customers who may not use the online system, ensuring accessibility and inclusiveness.

- **SMS Confirmation for Phone Orders:**
  When the customer service representative places an order, the system will send an SMS confirmation to the customer's phone number to confirm the order details.

- **Location-Based Delivery Agent Assignment:**

Implement a system that identifies delivery agents closest to the customer's location and displays a list of available agents. Customers or customer service representatives can then select the preferred delivery agent.

- **Delivery Agent Location Updates:**
  Enable delivery personnel to manually update their current location in the system or provide this information to customer service representatives for input, facilitating accurate delivery assignments.

- **Customer Location Specification:**
  Allow customers to specify their delivery location directly during order placement or through customer service representatives, ensuring accurate delivery routing.

➢ Stakeholders and their Roles

- **Client Company (Fresh Grocers Management):** The primary stakeholder responsible for defining business requirements, providing funding, and approving project deliverables. They oversee the project to ensure alignment with business goals and market demands.

- **Customers:** The main users of the system who place grocery orders online. Their satisfaction and ease of use are critical to the system's success. They provide feedback on system usability and delivery service quality.

- **Delivery Personnel:** Delivery agents who register on the system, update their location, and manage deliveries. Their interaction with the platform impacts the efficiency of order fulfilment.

- **Customer Service Representatives:** Staff members who assist customers by manually placing orders in the system and managing phone-based transactions. They act as intermediaries for customers who may not use the online platform.

- **Project Team:** The group responsible for planning, designing, developing, testing, and deploying the system. This includes:
    - ✓ ***System Developers and Software Engineers:*** Responsible for coding, implementing system features, and integrating functionalities as per requirements.
    - ✓ ***Quality Assurance (QA) Engineers:*** Handle system testing, identify bugs, and ensure the software meets quality standards before release.
    - ✓ ***UI/UX Designers:*** Design user-friendly interfaces ensuring the system is easy to navigate for all user groups.

- **IT Personnel (Internal Support Team):** Handle infrastructure setup, network configuration, and ongoing technical support post-deployment to maintain system stability and security.

- **Software Vendors/Third-party Providers:** External providers supplying software tools, SMS gateways, or cloud services required for functionalities such as automated notifications and hosting.

- **Finance Team:** Oversee the project budget, cost management, and financial approvals ensuring that resources are allocated efficiently.

- **Regulatory Authorities:** Ensure that the system complies with local data protection laws, electronic transaction regulations, and any other relevant standards.

➢ Feasibility Criteria

1. Operational Feasibility

- **Readiness of Existing Staff:**

  Fresh Grocers already has a structured team of customer service representatives and delivery personnel. These staff members are familiar with the current order and delivery workflows and will require only basic training to use the new system. For example, customer service representatives will learn how to manually enter customer orders through the system, while delivery agents will learn how to update their location and manage delivery assignments digitally. This ensures a smooth transition with minimal resistance or disruption.

- **Customer Willingness and Capability:**

  A significant portion of Fresh Grocers' customer base resides in city and suburban areas where internet usage and smartphone penetration are high. These customers are likely to welcome a more convenient way to order groceries online. The system also retains support for non-digital customers, ensuring no one is excluded, orders can still be placed via phone with assistance from a customer service representative, maintaining accessibility for all demographics.

- **Improved Efficiency and Service Quality:**

  Currently, many operations such as order taking, delivery coordination, and customer notifications are handled manually, which increases the risk of delays and errors. The new system automates these processes, ensuring faster order processing, accurate delivery assignments based on location, and real-time SMS updates. This will enhance customer experience and reduce the workload on staff, allowing them to focus on resolving issues and improving service quality.

- **Inclusivity and Flexibility:**

  One of the system's strengths is its flexibility. It accommodates both online and offline users. While customers who are comfortable with digital technology can use the web

platform, others can still place orders via phone. This inclusive approach ensures operational continuity for all customer segments during and after implementation.

- **Ongoing Maintenance and Support:**
  The system will be maintained by in-house IT personnel or third-party developers. This includes regular updates, bug fixes, and monitoring system performance. The organization's IT support capacity ensures that the platform will remain reliable and secure over time.

  In summary, the proposed system is highly compatible with Fresh Grocers' existing operational model. It enhances key areas such as order processing, delivery coordination, customer interaction, and staff productivity. With proper training and support, the system can be successfully integrated and operated within the organization, resulting in improved service delivery and long-term operational efficiency.

2. Technical Feasibility

- **Hardware Requirements**
  a) **Server Machine:** A dedicated server will be required to host the database and application backend. This server will manage all system processes, including order handling, user management, and communications with third-party services like SMS and map APIs. It should have sufficient RAM (at least 8–16GB), a multi-core processor, and SSD storage to ensure fast data retrieval and processing. The server must also have backup capabilities and uninterrupted power supply (UPS) support for reliability.
  b) **Branded Desktop PCs:** Medium-range branded desktop computers (e.g., Dell or HP) will be allocated for the use of customer service representatives and administrative staff. These machines will be used for tasks such as manually placing orders, accessing customer records, assigning delivery personnel, and generating reports. These brands offer durability, technical support, and compatibility with the proposed software stack, which reduces hardware-related risks.

c) **Printing Equipment:** An inkjet printer will be included in the setup for generating physical records such as receipts, order confirmations, or delivery summaries. While most operations are digital, printing capabilities ensure flexibility and documentation when required.

- **Network Infrastructure**
  a) **Local Area Network (LAN):** The office setup will require a secure LAN to connect the server with the desktop PCs. This setup allows real-time sharing of data such as order status, inventory changes, and customer queries between systems, ensuring efficient internal communication.
  b) **Internet Connectivity:** A stable and high-speed internet connection is necessary for enabling access to the web-based system by customers and delivery personnel. Internet access also allows the system to send SMS notifications and retrieve location data using APIs. It is essential that the connection is reliable to avoid service interruptions.
  c) **Networking Devices:** Basic networking hardware like routers, switches, and Ethernet cables will be used to establish connectivity between the server and client machines. These devices form the backbone of communication and ensure data is transferred efficiently and securely.

- **Software Requirements**
  a) **Development Environment:** The system will be developed using the C# programming language within Microsoft Visual Studio 2017 IDE. This environment supports rapid development, debugging, and integration with other Microsoft tools. It is highly suitable for enterprise-grade applications and well-documented, making maintenance easier.
  b) **Database Management System:** Microsoft SQL Server 2017 will be used to store user data, order history, delivery records, and other transactional information. It is a reliable and secure relational database system that ensures data integrity and supports concurrent transactions.
  c) **SMS and Location APIs:** The system will integrate with SMS gateway services (such as Twilio or a local provider) to send automatic notifications to customers after order placement. Additionally, integration with Google Maps API will

allow real-time proximity calculation between customers and delivery agents, enabling efficient agent selection.

d) **Web Interface Framework:** The web front-end may be developed using ASP.NET MVC or a similar framework that supports a responsive, clean, and user-friendly interface for all stakeholders, including customers, delivery personnel, and administrative staff.

- **Technical Support**
  a. **Experienced Development Team:** The project team has previous experience using the proposed tools (C#, SQL Server, Visual Studio). This minimizes technical learning curves and ensures that common challenges can be resolved quickly based on prior knowledge.
  b. **Availability of Local Resources:** All hardware and software required for the project are easily available in Sri Lanka. This means that there will be no delays in sourcing equipment or licenses, and local vendors can provide quick replacements or upgrades if needed.
  c. **Maintenance and Future Upgrades:** The system will be designed with scalability and maintainability in mind. Regular updates, bug fixes, and feature enhancements can be managed either by in-house IT personnel or outsourced software developers. Proper documentation and version control practices will be followed for smooth transitions.
  d. **User Training and Documentation:** To ensure ease of use, end users such as customer service agents and delivery personnel will be trained on how to use the system. Training sessions, user manuals, and quick-start guides will be provided. This guarantees effective adoption and reduces dependency on technical staff for daily operations.

3. Economic Feasibility

INITIAL COST	
Development cost	200,000
Hardware cost	350,000
Software License cost	50,000
Implementation and Deployment cost	30,000
Subscription fee (third-party services)	20,000
Total Initial cost: 650,000	

ONGOING COSTS (Every year)	
Electricity and IT infrastructure	100,000
Maintenance and Technical Support	120,000
Software Subscription renewal	30,000
Internet and Communication costs	40,000
System upgrades and enhancements	50,000
Security and Backup services	20,000
Total Initial cost: 360,000	

EXPECTED BENEFITS	
Increased Revenue	400,000
Cost Savings and Staff Reduction	200,000
Total Initial cost: 600,000	

Year	Cost (LKR)	Benefit (LKR)	Net Benefit (LKR)	Accumulated Net Benefit (LKR)
0	650,000	0	0	0
1	360,000	600,000	240,000	240,000
2	360,000	600,000	240,000	480,000
3	360,000	600,000	240,000	720,000
4	360,000	600,000	240,000	960,000

Payback Period

Payback Period= Initial Investment/ Net Annual Benefit = 650,000/240,000

≈ 2.7 years

This indicates the project recovers its costs in about **2 years and 7 months**, which is reasonable for such an investment.

Return on Investment (ROI)

Using Year 4 accumulated net benefit:

ROI = (Net Income / Cost of Investment) × 100 = (960,000 / 2090, 000) × 100 ≈ 45.93%

An ROI of approximately **46%** signals a strong financial return and supports moving forward with the project.

4. Schedule Feasibility

The client, Fresh Grocers, expects the web-based grocery delivery system to be fully developed and operational within **two months**. The development is scheduled to begin in the first week of July 2025, and the system is expected to go live by early September 2025**.** To determine if this timeframe is realistic, an effort estimation has been performed based on the required components of the system such as data entry forms and reports.

Effort Estimation

Component	Number Required	Lines of Code per Item	Total Lines of Code
Data Entry Forms	12 (Registration, Login, Order Placement, Location Entry, Order Assignment, etc.)	300	300 × 12 = 3,600
Reports	5 (Delivery Report, Customer Feedback, Daily Orders, Agent Ratings, Revenue Report)	100	100 × 5 = 500
			Total: 4,100 Lines of Codes

Time Allocation Chart

Phase	Estimated Time	Feasibility	Description
Requirements Analysis	5 working days	**High**	Stakeholder requirements are clear and well-defined, making this phase straightforward.
System Design	5 working days	**High**	Includes ER diagrams, UI wireframes, and workflow planning. Scope is small to medium scale, manageable within timeframe.
Coding & Unit Testing	7 working days	**Medium**	Requires implementation of approximately 4,100 lines of code. With two developers, this is realistic, but some complexity may arise.
User Acceptance Testing	2 working days	**High**	Since only key features will be tested by end-users, this short phase is feasible.

User Manual Preparation	4 working days	**High**	Standard documentation for basic user and admin functions. Time allotted is sufficient.
Buffer (Contingencies)	3 working days	**High**	Built-in buffer helps address unexpected issues such as bugs, feature tweaks, or delays.

- Total Time Required: 26 working days (Approximately 5.2 weeks)
- Time Available: 2 months (8 weeks)
- Considering a 5-day work week, some public holidays, and occasional sick leave, the project can realistically be completed in under 1.5 months. Since the client has allowed 2 full months, this schedule provides enough flexibility to manage risks and delays.
- **Conclusion:** With all phases rated mostly high in feasibility and a reasonable buffer included, the project is schedule feasible.

5. Legal Feasibility

Legal feasibility assesses whether the proposed system complies with relevant laws, regulations, and industry standards. For Fresh Grocers, the web-based grocery ordering and delivery system is considered legally feasible, based on the following considerations:

1. Data Protection and Privacy Laws: The system will handle sensitive customer data such as names, phone numbers, locations, and order details. Therefore, the platform must comply with Sri Lanka's Data Protection Act No. 9 of 2022, which mandates: Explicit user consent for data collection and usage, secure storage and processing of personal data, and allowing users to access, correct, or delete their data. Proper privacy policies and user agreements will be included in the system to ensure legal compliance.

2. Electronic Transactions: As the system supports online ordering and communication via email and SMS, it must comply with the Electronic Transactions Act No. 19 of 2006. This includes: Recognizing electronic communications as legally valid, ensuring transaction integrity and authenticity, and providing secure channels for communication between users and the system.

3. Employment and Labor Regulations: The platform involves registration and assignment of delivery personnel. Therefore, Fresh Grocers must ensure compliance with: Employment laws, including fair working conditions, working hours, and payment regulations for delivery agents, Proper contracts or service agreements must be in place if delivery agents are third-party contractors.

4. Consumer Rights and Fair Trading: The system must adhere to consumer protection laws such as: Providing accurate information on groceries, prices, and delivery times, ensuring fair billing practices and refund/cancellation policies, and enabling customer complaints and rating mechanisms.

5. Software Licensing: All development tools and software used (e.g., C#, MS SQL Server, Visual Studio) must be properly licensed. The use of open-source libraries must comply with their respective licenses (e.g., MIT, GPL).

Conclusion: The proposed system is legally feasible as long as it incorporates appropriate measures to protect personal data, ensure secure online transactions, follow employment standards, respect consumer rights, and use properly licensed software tools. Legal consultation may be done during development to ensure full compliance.

➢ Risk Assessment for the System

Risk	Description	Impact Level	Likelihood	Mitigation Strategy
Data Breach or Unauthorized Access	Sensitive customer and delivery data may be accessed by hackers or unauthorized users.	High	Medium	Use encryption, secure authentication, role-based access control, and conduct regular security audits.
System Downtime or Server Failure	Server or internet failures could render the system unusable.	High	Medium	Use reliable hosting, backup servers, regular maintenance, and monitoring systems.
Inaccurate Delivery Assignments	Outdated or incorrect location data may assign the wrong delivery agent, delaying delivery.	Medium	Medium	Ensure frequent location updates from agents and validate user input.
SMS and Email Delivery Failures	SMS or email messages with login details and confirmations might not be delivered.	Medium	High	Use trusted APIs, add retry mechanisms, and offer alternative in-platform notifications.
User Resistance to Adoption	Some users may be reluctant or find it difficult to use the new system.	Medium	Medium	Provide tutorials, a user-friendly design, and ongoing customer support.
Project Timeline Overruns	The system may not be completed by the scheduled deadline	High	Medium	Follow a clear project schedule, allocate buffer

	due to unforeseen delays.			days, and track progress regularly.
Cost Overruns	The project may go over budget due to unexpected feature requests or technical issues.	**Medium**	**Medium**	Monitor budget closely, define scope clearly, and prioritize essential features.
Legal or Compliance Issues	Failure to comply with data privacy and digital transaction laws may result in penalties.	**High**	**Low**	Consult legal advisors and ensure compliance with all applicable regulations during development.

While risks such as data breaches, system unavailability, and deadline for the project overruns carry considerable effect, the possibility of these situations can be decreased with robust security protocols, reliable infrastructure, and smart project management. Medium-level risks, including improper delivery assignments, communication failures, and user resistance, can be mitigated by assuring timely updates, employing reliable communication services, and developing an intuitive user interface with enough support. Although legal compliance concerns exist, they are rather low in likelihood and may be addressed by consulting legal specialists throughout development. Considering the thorough identification of these risks and realistic solutions, the project team is well-prepared to tackle potential hurdles. This balanced understanding and preparedness imply that the system is suitable for development. The predicted benefits of streamlining operations, enhancing customer experience, and improving delivery coordination outweigh the controllable risks, making the proposed system a practical and strategically useful investment for Fresh Grocers.

➢ Alternative Technical Solutions with Cost/Benefit Analysis

Given the outlined cost/benefit analysis, several alternative technical solutions can be considered to improve cost-effectiveness while maintaining or enhancing system performance. One option is to utilize open-source technologies instead of licensed software. For instance, replacing paid tools like Microsoft Visual Studio and SQL Server with open-source alternatives such as Visual Studio Code and PostgreSQL could significantly reduce software licensing costs. This shift would lower the initial software cost and eliminate annual subscription fees, thereby improving the system's ROI and shortening the payback period.

Another potential solution is adopting a cloud-based software-as-a-service (SaaS) model instead of building a fully customized system from scratch. This approach could reduce development and maintenance costs since cloud platforms often offer built-in security, scalability, and infrastructure management. It may also eliminate the need for in-house servers and reduce electricity and IT infrastructure costs. Cloud platforms like Amazon Web Services (AWS), Microsoft Azure, or Google Cloud could provide scalable options with pay-as-you-go pricing models that align with the growth of Fresh Grocers.

Additionally, outsourcing certain components of the development or technical support to third-party vendors could reduce internal staffing costs and speed up deployment. Hybrid app development using cross-platform frameworks such as Flutter or React Native could also reduce development time and cost if mobile app support is later considered.

These alternatives, when evaluated against the existing economic feasibility data, can potentially reduce the initial and ongoing expenses, increase the net benefit, and deliver a faster payback period with an even stronger return on investment.

➢ Recommendations

Based on the comprehensive feasibility study, it is strongly recommended that Fresh Grocers proceed with the development and implementation of the proposed web-based ordering and delivery system. To ensure success, the company should adopt a development approach that balances reliability and cost-effectiveness, potentially exploring both proprietary and open-source technologies to optimize long-term expenses. Utilizing cloud-based infrastructure is also advisable, as it can reduce upfront hardware costs, enhance scalability, and improve data security and system maintenance. The system should incorporate role-based access control to protect sensitive information and ensure proper user permissions for customers, delivery agents, and staff. Key operational tasks such as customer registration, order placement, delivery agent assignment, and SMS notifications should be automated to improve efficiency and reduce manual errors. Additionally, a strong plan for ongoing maintenance and technical support must be established to maintain system stability and address issues promptly. Comprehensive training programs for staff and delivery personnel will be essential for smooth adoption and effective use of the system. Once deployed, continuous monitoring of performance indicators and user feedback should be implemented to guide ongoing improvements. Finally, the system architecture should be designed with future scalability in mind, allowing Fresh Grocers to incorporate additional features such as mobile app support and online payment options as the business grows. Following these recommendations will help ensure the system delivers maximum value and supports the company's long-term growth and competitive advantage.

The End

2.2 Analyzing each feasibility criteria identified and determining whether they are relevant to the problem statement

The feasibility of the proposed web-based grocery ordering and delivery system for Fresh Grocers has been thoroughly tested against the key components of the problem statement. The primary concerns emphasized in the problem statement include the limitations of manual processes, lack of an online ordering platform, inadequate communication between customers and delivery agents, and the lack of automated service features such as SMS notifications, customer ratings, and location-based delivery assignments. Evaluating the system via operational, technological, economic, scheduling, and legal perspectives indicates how well the proposed solution solves these challenges.

Operational feasibility is tightly aligned with the demands indicated in the problem statement. The current approach is mostly manual and lacks digital convenience for both clients and internal workers. By incorporating online registration, real-time order placement, and automated communication, the new system delivers solutions that are easy to use and well-suited to the workflow of customers, delivery agents, and customer care workers. Moreover, the willingness of users to adopt technology, especially in a period of increased internet usage, enhances the practical feasibility of the project. The solution corresponds nicely with the goal of enhancing customer experience and improving operational efficiency.

Technical feasibility also supports the adoption of the suggested system. The problem statement explicitly calls for capabilities such as online user registration, SMS notifications, location tracking, and database-driven order management, all of which are possible using publicly accessible technology. Using proven development tools like as C#, Microsoft SQL Server, and a Windows-based platform assures that the solution can be constructed using existing skills and infrastructure. The availability of skilled developers and proven tools means the technical needs can be accomplished without introducing excessive complexity or risk.

Economic feasibility is another significant area where the suggested system displays alignment with the problem statement. The highlighted inefficiencies in the present system such as manual order entry and lack of digital communication, lead to greater operating

costs and restricted consumer involvement. The new system is expected to deliver considerable financial benefits, including higher income through a bigger client base and cost savings through staff workload reduction. With a fair payback period of roughly 2.7 years and a return on investment of 46%, the economic evaluation clearly supports the installation of the system as a financially sound solution to the challenges outlined.

Schedule feasibility is also directly tied to the urgency and expectations mentioned in the problem. Fresh Grocers hopes to capitalize swiftly on the expanding trend of online grocery buying. The development timetable of two months, as asked by the client, is reasonable and achievable based on effort estimation and the scope of required features. Proper planning and resource allocation ensure that the project can be finished within the allotted timeline, allowing the business to obtain a timely competitive edge.

Finally, legal feasibility validates that the system will work within the limitations of applicable laws and regulations. The problem statement includes the processing of customer data, SMS communication, and user accounts, all of which require compliance with data privacy regulations and communication standards in Sri Lanka. Ensuring that the system adheres to these legal duties protects the company from potential legal issues and creates trust among its users.

In conclusion, the five feasibility criteria; operational, technical, economic, timetable, and legal, are highly significant and supportive in addressing the difficulties described in the problem statement. Together, they indicate that the proposed web-based solution is not only viable and achievable but also strategically linked with Fresh Grocers' goals of modernizing its operations and boosting customer service.

Activity 03

3.1 Strengths and Weaknesses of Traditional Methodology

❖ Strengths

- **Structured and Consistent Procedure:**

Conventional techniques adhere to a rigid linear sequence, necessitating the completion of each phase prior to the commencement of the subsequent one. This predictability enables managers to plan resources, timeframes, and budgets with great confidence. It is advantageous when requirements are clearly defined and unlikely to fluctuate. For instance: Creating a payroll system for a governmental agency, wherein regulations, procedures, and reporting standards are predetermined and formally defined in advance.

- **Comprehensive Documentation**

Each phase is meticulously documented, encompassing requirement descriptions, design blueprints, test plans, and user manuals. This fosters transparency, facilitates audits, and simplifies the on boarding of new team members or the long-term maintenance of the system. For the establishment of a national healthcare management platform, comprehensive documentation guarantees uniform adherence to protocols by all hospitals and clinics.

- **Defined Milestones and Responsibility**

Each phase possesses distinct deliverables and dates, facilitating progress monitoring and team accountability. Managers may analyse project health at each stage without overlapping issues. For example; A student information system established for a university can follow explicit checklists such as “requirements gathered,” “system designed,” “modules coded,” etc.

- **Lower Client Involvement during Development**

Clients are highly involved at the beginning and conclusion but not throughout, which suits those with limited availability or technical competence. For instance; a logistics company

may submit early specs for a fleet management tool and return only at final testing and delivery.

- **Ease of Budgeting and Scheduling**

Since all criteria and plans are specified before development begins, cost and time estimates are more precise, lowering the possibility of surprises mid-project. For example; A bank commissioning software to automate loan approvals can budget its IT department resources effectively using Waterfall.

- **Comprehensive System Testing**

Testing at the end allows teams to evaluate the system in its totality, ensuring all components perform seamlessly together. For example: In a large-scale ATM network software update, full-system testing helps eliminate compatibility or transaction issues across branches.

- **Suitable for Regulatory and Safety-Critical Systems**

Traditional techniques like Waterfall are particularly effective for sectors that demand tight regulatory compliance, where every process must be carefully documented, evaluated, and signed off before moving forward. The linear design assures transparency and accountability at every stage, which is critical for projects involving safety, healthcare, or aerospace systems. For instance, in aviation software, such as autopilot control systems or airport traffic monitoring tools, regulatory organizations like the FAA need complete documentation, testing, and approval. A strict Waterfall approach ensures nothing is left undocumented or incorrectly tested before deployment.

❖ Weaknesses

- **Rigid Structure**

  Once a phase is done, returning it is difficult and costly. The approach implies requirements are unchanging, which is rarely the case in real-world projects. For example if customers of a real estate management system later ask for map integration after design is finalized, it may entail rewriting the entire UI.

- **Late Testing and Feedback**

  Testing is done only after implementation, delaying the identification of flaws or user experience issues. For instance, a mobile banking app may reach the testing phase before recognizing customers struggle with navigation or performance on lower-end phones.

- **User Involvement Is Minimal**

  Limited feedback during development means the ultimate product may not match evolving user demands or expectations. For instance, teachers might not be contacted during creation of an online grading platform, resulting in features they never use.

- **High Risk of Failure from Early Errors**

  Mistakes created in the requirements or design phase are carried throughout the project, frequently unrecognized until it's too late. Misunderstanding how a merchant monitors inventory could lead to a system that interrupts everyday operations.

- **Slow Delivery Time**

  The product is only supplied at the end, resulting in significant wait times before stakeholders see any working features. For instance, a ride-hailing business utilizing Waterfall may take 12 months to create an app, during which competitors release speedier alternatives.

- **Resource Wastage if Project Fails Midway**

  Due to the all-or-nothing nature, incomplete projects may provide no useable components. For example, a public transportation scheduling platform cancelled after a year of planning and design ends with no operating system.

- **Low Adaptability in Fast-Paced Markets**

  In fast-evolving businesses like e-commerce or social media, client expectations vary rapidly. Traditional methods can't accommodate such rapid changes mid-process, sometimes leading to solutions that are outdated by the time they're completed. By the time the finished product is launched, competitors may have already responded to new trends. A fashion e-commerce company adopting Waterfall might spend 12 months constructing a site, only to find that client preferences for AI-powered personal shopping or influencer integration have become the new norm, rendering their system outmoded.

3.2 Strengths and Weaknesses of Agile Methodology

❖ Strengths

- **High Flexibility to Changes**

Agile allows evolving requirements even when they are late in development. Iterative sprints make it easier to modify priorities or add features based on market feedback. For instance, a fashion retailer's e-commerce site can swiftly introduce new filters or payment choices depending on user ideas.

- **Frequent Customer Involvement and Feedback**

Clients are regularly involved, assessing each sprint's deliverables and ensuring the system fits with expectations. For example, a food delivery start up tries real-time location monitoring with users and modifies based on delivery driver input.

- **Faster Delivery through Iterations**

MVPs (Minimum Viable Products) and working features are delivered often, enabling speedier time-to-market and early return on investment. Examples include a new technology company produces a rudimentary event ticketing tool within weeks, adding QR code scanning and analytics in future sprints.

- **Encourages Collaboration and Teamwork**

Agile creates a shared sense of ownership through daily stand-ups, retrospectives, and open communication among developers, testers, and product owners. An instance of a health-tech start up uses cross-functional teams to design a patient tracking app where doctors, IT, and administration all contribute ideas.

- **Higher Product Quality through Continuous Testing**

With continued testing in each sprint, faults are found and corrected early, ensuring greater overall quality. For instance, a SaaS accounting platform tests each new dashboard component as it is produced, eliminating problems at final delivery.

- **Higher Customer Satisfaction**

Regular releases and prompt answers to feedback result in a system that closely meets user expectations. For instance, a trip booking app adds preferred airline filters and loyalty rewards features based on early user ratings.

- **Promotes Continuous Learning and Improvement**

Agile includes retrospectives at the end of each sprint, where teams analyse what worked well and what didn't. This generates a culture of constant learning, process refinement, and performance improvement, which increases team dynamics and product quality over time. For instance, in designing a meal delivery app, the team may find in a sprint retrospective that push messages are delivered too frequently and anger users. They then update their notification logic in the next sprint based on statistics and feedback.

❖ Weaknesses

- **Difficult to Predict Cost and Schedule**

Due to continuous modifications, it's hard to anticipate final pricing and delivery timeframes. This volatility might frustrate stakeholders. Examples include a budgeting application project initially scoped for 6 months may stretch to 10 owing to increasing feature sets.

- **Risk of Scope Creep**

Continuous requirement increases without sufficient bounds might lead to bloated features and longer timelines. For example, fitness app ends up incorporating social media integration, calorie tracking, and coaching modules far beyond the intended aim.

- **Limited Documentation**

Agile stresses functioning software over documentation, which can cause difficulties for maintenance teams or regulatory audits. For instance, a financial system missing sufficient records or interface maps becomes challenging to maintain throughout personnel turnover.

- **Requires High Client Involvement**

Clients must attend regular meetings and evaluations, which is not always possible for busy or non-technical stakeholders. For instance, a real estate manager may not have time to attend weekly reviews for the new CRM being constructed.

- **Demands Skilled Teams**

Agile depends on self-organizing, cross-functional teams. Inexperienced teams may struggle with prioritization and coordination. For example, junior engineers may overpromise and under deliver during sprints in a hospital management software project.

- **Harder to Scale across Large Enterprises**

Agile becomes complex when several teams need to align their sprints and merge components regularly. For instance, a multinational bank's app development in four countries may face version mismatches and delays.

- **Challenging to Document Long-Term Vision**

While Agile thrives at short-term iteration, it can be challenging to maintain a consistent long-term vision, especially when priorities vary regularly. Without a good roadmap or future architecture plan, the program can become disconnected or too complex over time. For example, in constructing a modular content management system (CMS), repeated changes in sprint goals could lead to inconsistent data models or redundant code, making the system harder to maintain in the long term.

3.3 Strengths and Weaknesses of Feasibility Study

❖ Strengths

- **Reduces Risk through Early Evaluation**

A feasibility study investigates technical, operational, legal, and economic aspects before development begins. This decreases the chance of unanticipated difficulties. As an instance, a delivery app team determines in their analysis that GPS tracking APIs are pricey and plan appropriately.

- **Better Stakeholder Buy-in**

Decision-makers are more likely to support the initiative if they see data-backed evidence of success potential and ROI. For example, a grocery chain advances with a self-checkout system after feasibility demonstrates 2.7-year payback and substantial customer demand.

- **Improves Scope and Planning Accuracy**

Identifying needs, restrictions, and expectations early ensures reasonable scope and timeframe forecasts. For instance, a feasibility assessment for a digital school attendance system demonstrates the requirement for offline mode in remote schools.

- **Legal and Ethical Safeguards**

Legal feasibility assures compliance with requirements including data protection legislation, preserving the organization from penalties. A good example is telemedicine app alters its design after the study indicates it must comply with HIPAA and Sri Lanka's ICTA rules.

- **Aids in Resource Allocation**

Feasibility studies assist deploy developers, designers, funding, and infrastructure effectively, eliminating under or over-investment. For instance, a start-up realizes it only needs one backend developer during the MVP phase, saving money.

- **Helps Choose Development Methodology**

The study may advise Waterfall for stable conditions or Agile where flexibility is needed. For example, an e-learning platform with shifting content requirements is better suited to Agile as demonstrated by feasibility study.

- **Enhances Stakeholder Communication**

Feasibility studies operate as a bridge between technical teams, management, investors, and other stakeholders. By delivering a clear, objective assessment of viability across several domains (technical, economic, legal), they guarantee all parties understand the project scope, restrictions, and expectations from the outset. For instance, a company planning to create a crypto currency payment gateway utilizes a feasibility analysis to highlight potential regulatory limits and market adoption rates, aligning both funders and the dev team on feasible targets.

❖ Weaknesses

- **Time-Consuming**

  The study process incorporates surveys, interviews, financial forecasts, and research, which might delay growth. For instance, a three-week feasibility phase may force a retail business to miss a Christmas season launch.

- **Can Be Expensive**

  Hiring specialists and gathering data may cost large amounts, which may be unjustified for small or low-risk enterprises. For instance, a small online bakery might not benefit from a LKR 100,000 feasibility study.

- **Based on Assumptions**

  Inaccurate data or assumptions might bias the outcomes, leading to incorrect conclusions. For instance, assuming infinite 4G access in remote locations could mistakenly validate a mobile-only delivery platform.

- **Not a Guarantee of Success**

  Even if possible on paper, poor execution, team difficulties, or market shifts can lead to failure. For instance, a well-planned taxi app fails owing to lack of driver acceptance and severe competition.

- **May Limit Innovation**

  The feasibility phase may focus too heavily on risk minimization and discourage creative ideas. For instance, a study suggested AI chatbot for healthcare may be abandoned due to high early cost estimates, despite long-term benefits.

- **Short Shelf Life of Data**

  In fast-changing businesses, the findings can become outdated soon. For instance, a pricing model that was realistic six months ago for cloud services may no longer apply due to inflation or supplier changes.

- **May Discourage Risk-Taking or Innovation**

  When feasibility studies are unduly focused on risk and financial return, they can foster a conservative attitude where innovative or unusual ideas are disregarded owing to perceived uncertainty or lack of immediate ROI, even if they offer long-term strategic value. For instance, a feasibility analysis may propose abandoning an AI-driven customer service bot because of initial training expenses and unknown accuracy, despite its potential to lower customer service costs dramatically

3.4 Evaluation of Methodologies and Feasibility Study Effectiveness against the Problem Statement

Fresh Grocers, a grocery delivery service in Sri Lanka, currently works with a manual system that relies mainly on telephone calls for order placement. This outmoded paradigm lacks automation, digital tracking, client interaction capabilities, and streamlined communication, making it ineffective in fulfilling expanding customer needs. To modernize its processes, Fresh Grocers plans to implement a software-based solution. Evaluating the strengths and shortcomings of both Traditional and Agile techniques, together with the feasibility study, against this problem statement, is vital to determine their success in driving a suitable solution.

Traditional approaches, such as the Waterfall model, include strengths like precise documentation, well-defined stages, and predictable scheduling, which are effective in projects with established requirements. In the case of Fresh Grocers, these strengths could enable systematic planning and compliance with industry norms, especially in managing sensitive client data or delivery schedules. However, given the dynamic nature of food delivery services, where consumer wants and market trends change rapidly, the rigidity of the Waterfall approach becomes a severe drawback. The inability to adapt modifications once production begins implies any new feature requests like the addition of live tracking or digital payment options would be costly and time-consuming to implement. This incompatibility with the flexible demands of Fresh Grocers' target market decreases the efficiency of the traditional approach in fixing the situation at hand.

Agile approaches, on the other hand, correlate more closely with Fresh Grocers' requirement for adaptation and speed. Agile's strengths; iterative development, frequent customer input, and early delivery of functional modules directly address the inefficiencies in Fresh Grocers' manual approach. For instance, a digital ordering system can be constructed and distributed in stages, beginning with a basic ordering capability and subsequently adding inventory tracking, customer evaluations, and payment connections. This iterative method helps the organization to respond to consumer feedback, test features in real-time, and make speedy adjustments, all crucial in a competitive delivery market. Nevertheless, Agile's limitations, such as difficulty in projecting costs and scope creep, can

offer obstacles, especially if the project team lacks experience or if stakeholders are unable to engage regularly. In Fresh Grocers' environment, this could result in delays or misplaced priorities if the development is not carefully controlled.

The feasibility study works as a critical bridge in this evaluation. It gives an early assessment of the technological, economic, legal, and operational viability of the proposed system. In Fresh Grocers' case, the feasibility study helps establish whether the company has the infrastructure to support a digital platform, examine the cost-benefit ratio of automation, and assure compliance with data protection rules. The power of the feasibility study resides in decreasing ambiguity and delivering educated estimates that aid in planning and resource allocation. For example, if the feasibility research finds that most of the consumer base owns smartphones and enjoys digital engagement, it supports investing in a mobile app. However, the constraints of feasibility studies such as their reliance on assumptions and ability to become outdated quickly, can restrict their long-term utility. If the market or technology trends evolve rapidly, an earlier feasibility assessment may not completely reflect new realities, potentially misleading project decisions.

In conclusion, the research demonstrates that while traditional techniques give structure, their limits in handling change and innovation make them less successful for meeting the special demands of Fresh Grocers. Agile techniques, albeit requiring careful management, offer more alignment with the company's goals of flexibility, user involvement, and gradual growth. Meanwhile, the feasibility study appears to be a significant beginning tool in directing decision-making and risk assessment, while its efficacy depends on ongoing updates and validation. Overall, Agile, accompanied by a thorough and adaptive feasibility analysis, looks more effective in addressing the problem statement.

3.5 Recommendations on the Most Suitable Methodology Based on Identified Strengths and Weaknesses

Given the examination of strengths and weaknesses of Traditional and Agile techniques, as well as the significance of feasibility studies, it becomes necessary to recommend the most effective option for Fresh Grocers. The company's ambition to migrate from a manual, phone-based ordering system to an automated, digital platform necessitates a methodology that enables rapid development, adaptability, and user-centered design. Based on these requirements, Agile approach appears as the most suitable alternative, supported by an adaptive feasibility assessment.

Agile's iterative nature enables for continual delivery of features and fast incorporation of user feedback. This is particularly helpful for Fresh Grocers, as consumer preferences for things like live tracking, mobile app integration, or payment channels may alter over time. Unlike the Traditional Waterfall methodology, Agile does not require all requirements to be established upfront. Instead, it accommodates change throughout the project lifecycle, enabling Fresh Grocers to remain competitive in a changing market. This responsiveness ensures that the final product aligns closely with both company objectives and user expectations.

Furthermore, Agile's focus on cooperation across cross-functional teams and stakeholders guarantees that development is transparent and user-focused. Regular sprint reviews and retrospectives enable shared responsibility, problem-solving, and ongoing progress. In the context of Fresh Grocers, including both technical workers and business stakeholders ensures that crucial insights such as logistics issues or customer pain points are absorbed into the system as it matures. This boosts the relevance and utility of the final output.

While Agile has obvious disadvantages, such as the risk of scope creep and the necessity for frequent stakeholder interaction, they can be avoided by excellent project management methods. Clear sprint goals, prioritizing of backlog items, and disciplined use of technologies like Scrum or Kanban can assist keep the project focused and within budget. Additionally, with sufficient training and team structuring, Fresh Grocers can create the self-organizing teams that Agile relies on for successful delivery.

In contrast, while Traditional techniques like Waterfall offer characteristics like as organized documentation, milestone clarity, and ease of compliance, their inflexibility makes them ill-suited for Fresh Grocers. The sequential nature of Waterfall reduces reactivity, which can be negative in a service-based firm that needs to continuously adjust to market trends and user expectations. For instance, if a demand for same-day delivery emerges after the design process, the cost and time required to retrofit this into the system could derail the project. Moreover, the delayed feedback loop inherent in Traditional methods raises the danger of providing a final product that is outdated or misaligned with consumer needs.

The feasibility study complements the Agile methodology by serving as a basic guide at project initiation. It provides important insights into the project's viability, enabling Fresh Grocers identify potential risks, estimate budget and resource requirements, and align goals. When connected with Agile, the feasibility study can be revisited and updated at important project milestones, ensuring its relevance throughout the development process. This dynamic usage of feasibility studies transforms them from static reports into living records that help ongoing decision-making.

Therefore, the recommended solution is to adopt Agile methodology as the primary project management framework, supported by an iterative feasibility research process. This combination enables Fresh Grocers to maintain agility while making educated, strategic decisions at each step of development. It allows the organization to produce a solution that is not only technically feasible and commercially viable but also sensitive to user wants and market circumstances.

In conclusion, the dynamic and customer-centric nature of Fresh Grocers' service environment necessitates an approach that embraces change, enables cooperation, and provides value progressively. Agile, supported by an adjustable feasibility assessment, fits these criteria effectively. With the proper implementation structure, this strategy guarantees the maximum possibility of success in transitioning Fresh Grocers from a manual operation to a modern, efficient, and competitive digital platform.

3.6 Proposed Solution for the Problem Statement

In response to the operational challenges and inefficiencies of Fresh Grocers' old manual ordering system, the company has identified the need for a digital shift through the development of a web-based grocery ordering and delivery management platform. With over fifteen years of expertise in serving consumers across city and suburban areas in Sri Lanka, Fresh Grocers has developed a firm foundation of customer trust. However, to remain competitive and fulfil the increased expectations of tech-savvy consumers, a more automated and accessible ordering mechanism is required. The suggested solution fulfils these expectations by establishing a consolidated, user-friendly digital platform that promotes productivity, service accuracy, and customer happiness.

The fundamental purpose of this solution is to replace the manual, phone-based process of placing and managing grocery orders by replacing it with a streamlined, digital interface. Through the new platform, clients and delivery people will be able to register online. Upon registration, users will receive a unique username and password via email, ensuring safe access to the system. This secure login procedure is vital for ensuring user privacy and securing critical information. By giving direct access, the technology empowers users to control their own activities while lowering the pressure on customer service professionals.

One of the most significant innovations brought by the new system is the ability for customers to place purchases directly through the website. Once an order is verified, the system immediately produces and sends an SMS to the consumer, which includes the contact details of the assigned delivery agent and an anticipated delivery time. This feature replaces the manual communication previously required, providing clients with quick clarity and real-time information regarding their order, thus dramatically boosting the transparency and speed of the service.

The system further provides a dynamic location-based assignment module, which automatically identifies delivery agents who are geographically nearest to the customer. This optimization not only improves delivery time but also decreases operational expenses by decreasing trip distances. Customers, or customer service personnel acting on their behalf, are given the option to select from the list of possible nearby delivery agents. This

adaptability facilitates both digital and manually aided order placements, making the platform inclusive for all client demographics, even those less comfortable with technology.

For delivery persons, the platform enables manual updating of their current location, which can also be entered by customer service staff when necessary. This ensures that the system's location data stays correct, even in the absence of automated GPS tracking. Meanwhile, customer support staff are prepared with tools to assist consumers who still prefer placing orders by phone. This human-centred support methodology ensures that Fresh Grocers does not alienate any of its existing client base while shifting to digital operations.

To ensure service quality, the system features a feedback feature that allows consumers to grade delivery staff after each delivery. This constant feedback loop is vital for performance monitoring and helps Fresh Grocers identify strengths and areas for improvement within its team. By establishing accountability and transparency, this element enables long-term consumer happiness and service development.

The suggested system incorporates all these features into a unified and intelligent platform that digitizes the entire supermarket ordering and delivery procedure. From registration and order placement to delivery assignment, real-time communication, and feedback gathering, every step of the service process is optimized for digital efficiency. By minimizing the dependency on manual procedures and introducing user-centric features, the system provides a fast, dependable, and modern service experience for clients, while also giving the organization with tools for increased operational supervision.

In conclusion, Fresh Grocers' proposed web-based grocery ordering platform represents a dramatic shift from a traditional, manually run service model to a technologically advanced and customer-centric one. It not only overcomes the shortcomings of the current system such as delays, miscommunication, and lack of transparency, but also offers a scalable framework for future expansion. With greater convenience, better resource management, and a seamless customer experience, this solution equips Fresh Grocers to succeed in Sri Lanka's quickly evolving digital economy.

3.7 User Requirements of the System

3.7.1 Functional Requirement

- **User Registration and Authentication**
  - ✓ The system must allow customers and delivery staff to register online.
  - ✓ Upon registration, users must get an email including a unique username and password.
  - ✓ The system must allow registered users to log in securely using their credentials.

- **Customer Order Placement**
  - ✓ Customers must be able to browse and select groceries via the website.
  - ✓ Customers must be able to place grocery orders through the system interface.
  - ✓ Upon order confirmation, the system must send an SMS to the customer with:
    - The allocated delivery agent's contact information
    - The estimated delivery time

- **Order Placement by Customer Service Representative (CSR)**
  - ✓ The CSR must be able to place grocery orders on behalf of consumers without logging in as them.
  - ✓ When placing such orders, the CSR must:
    - Enter the customer's phone number and location
    - Ensure the system sends an SMS confirmation to that phone number

- **Delivery Agent Assignment**
  - ✓ The system must be capable of determining the closest delivery agents depending on client or CSR input location.
  - ✓ When an order is placed, the system must:
    - Display a list of nearby available delivery agents
    - Allow the customer or CSR to choose the preferred agent from the list

- **Location Management**
  - ✓ Delivery agents must be able to manually update their location through the system.
  - ✓ Delivery agents may also advise the CSR of their location, who can input it on their behalf.
  - ✓ Customers must specify their location during order placing, either by entering it directly or through the CSR.

- **Delivery Rating**
  - ✓ After each delivery, the system must allow clients to rate the delivery person based on their delivery experience.
  - ✓ These ratings must be saved in the system and correlated with the delivery agent's profile.

3.7.2 Non-Functional Requirements

- **Performance and Scalability:**

✓ The system must be capable of handling simultaneous user activity, including multiple customers making orders and delivery agents updating locations.

✓ The platform should facilitate growing as the number of user increases throughout cities and suburban regions.

- **Security:**

✓ All user data (including passwords and contact information) must be saved securely using encryption.

✓ User authentication should be controlled via secure login methods (e.g., HTTPS, hashed passwords).

✓ The system must check user input to prevent injection attacks or unauthorized access.

- **Reliability and Availability:**

✓ The platform must maintain high availability with little downtime, especially during peak grocery shopping hours.
✓ SMS and email notifications must be sent reliably and promptly.

- **Usability:**

✓ The user interface should be intuitive and user-friendly for all types of users, including non-tech-savvy customers.
✓ The CSR interface should facilitate efficient manual entry and easy navigation.

- **Maintainability:**

✓ The system should be constructed using a modular architecture, allowing quick modifications to individual components (e.g., rating system, SMS notifications).
✓ It should be straightforward for developers to add new features or modify current ones without affecting functionality.

- **Portability and Compatibility:**

✓ The system should be browser-independent and mobile-responsive so that users can access it from any device.
✓ Integration with external SMS and email APIs must be supported and simply configured.

- **Auditability and Logging**

✓ The system must maintain logs for:
  - Order placements
  - Location updates
  - CSR interactions
  - Ratings submitted

✓ These logs should be accessible to authorized administrators for auditing purposes.

- **Localization and Notification Support:**

✓ SMS and email content should offer configurable templates, potentially in Sinhala or Tamil for regional customers.

- ✓ Notification techniques should be customizable and scalable, e.g., using third-party gateways like Twilio.

3.8 System Requirements

- **Hardware Requirements**

  These requirements outline the physical infrastructure needed to host and operate the system, including server and client devices.

A. Server-Side Hardware (for Hosting the Web Application)

- Processor: Intel Xeon or AMD Ryzen Threadripper, 2.5 GHz or higher
- RAM: Minimum 8-16 GB (32 GB recommended for future expansion)
- Storage: 1 TB SSD (for speed and performance)
- Network Interface: High-speed internet connection with at least 1 Gbps bandwidth.
- Backup System: External backup disks or cloud-based backup service

B. Client-Side Hardware (Users' Devices)

- Device Types: Desktop, Laptop, Smartphone, Tablet
- Processor: Minimum Dual-Core 1.6 GHz
- RAM: Minimum 4 GB for desktops/laptops; 2 GB for mobile
- Storage: 512 MB free space (for cache and temporary storage)
- Connectivity: Internet connection (3G/4G/Wi-Fi)

- **Software Requirements**

  These include the operating systems, platforms, and tools necessary for development, deployment, and operation of the system.

A. Server-Side Software

- Operating System: Ubuntu Server 20.04 LTS or Windows Server 2019
- Web Server: Apache or Nginx
- Database: MySQL, PostgreSQL, or MongoDB

- Programming Language/Framework:
- Backend: PHP (Laravel), Node.js, or Python (Django)
- Frontend: HTML5, CSS3, JavaScript (with React or Angular)
- SMS & Email API Integration: Twilio for SMS, SendGrid or Mailgun for email

B. Client-Side Software

- Web Browser: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari (latest versions)
- Operating System Compatibility:
- Windows 10 or higher
- Android 9.0 or higher
- iOS 12 or higher
- macOS 10.13 or higher

C. Development Tools (For Developers)

- IDE: Visual Studio Code, PhpStorm, PyCharm, or WebStorm
- Version Control: Git with GitHub or GitLab
- Testing Tools: Postman (API testing), Selenium (automation testing)
- Project Management: Jira, Trello, or GitHub Projects

- **Security Requirements**

✓ Secure Authentication Mechanism: Password hashing (e.g., bcrypt), Two-Factor Authentication (2FA) optional
✓ SSL Certificate: HTTPS support via TLS 1.2 or higher
✓ Data Encryption: End-to-end encryption for sensitive data
✓ Access Control: Role-based access for customers, delivery personnel, and administrators
✓ Firewall & Anti-DDoS: Protection mechanisms at the server level

- **Network Requirements**

✓ Bandwidth: Minimum 100 Mbps for server hosting; scalable depending on user traffic
✓ Latency: Low-latency support (<150ms for API responses)

- ✓ Firewall Configuration: Secure open ports (e.g., 443 for HTTPS, 80 for HTTP, 22 for SSH)

- **Testing & Deployment Requirements**
- ✓ Test Environment: Separate staging server identical to the production server
- ✓ Continuous Integration (CI/CD): Jenkins, GitHub Actions, or GitLab CI/CD
- ✓ Containerization (Optional): Docker for isolated environments
- ✓ Monitoring Tools: New Relic, Prometheus, or Nagios for performance and uptime monitoring

- **Backup and Recovery Requirements**
- ✓ Automatic Backup Schedule: Daily database and weekly full-site backups
- ✓ Disaster Recovery Plan: Failover server or cloud-hosted mirror
- ✓ Data Retention Policy: Define a 6-month to 1-year customer and order data retention period

3.9 Reviewing the Requirement with fact finding techniques

To comprehensively analyse the user requirements for the Fresh Grocers web-based grocery ordering and delivery system, different fact-finding techniques will be performed to guarantee that all stakeholder demands are accurately recorded and the system design fits with real-world expectations. First, **interviews** will be done with key stakeholders, including customers, delivery agents, customer service representatives (CSRs), and system administrators. These interviews will help unearth specific needs, such as customers demanding features like SMS notifications with delivery information, the ability to explore and place purchases simply, and secure registration processes. Delivery agents will be interviewed to collect insights on requirements such location updating, order assignment preferences, and customer feedback. CSRs will provide input on the need to place orders on behalf of clients and manually update delivery agent positions when needed.

In addition to interviews**, questionnaires and surveys** will be given to a larger range of consumers and delivery agents to acquire quantifiable data on user behaviour, preferences, and frequent difficulties. These tools will allow the project team to uncover patterns, such as a strong demand for multilingual support in SMS communication, or user

expectations about system responsiveness and ease of navigation. The data will assist prioritize features and influence usability design choices.

Observation will also be employed as a technique, especially to understand how CSRs and delivery agents interact with the current manual processes. Observing how orders are entered, assigned, and completed in real time will expose inefficiencies, such as communication delays or data entry errors, which the new system must address. This will provide personal insight into workflows, allowing the system to be adjusted to genuine operational demands.

Furthermore, **document analysis** will be undertaken to evaluate existing records, including order logs, delivery sheets, customer feedback notes, and communication records. This study will assist uncover holes in the current process, such as missing tracking data or the absence of performance logs, and promote the introduction of features like audit trails, order history, and performance ratings.

Prototyping will be utilized to construct preliminary versions of the system interfaces, such as the customer ordering portal, CSR dashboard, and delivery agent panel. These mock ups will be presented to stakeholders for input, allowing them to visualize system functionality and provide suggestions for improvement. This iterative feedback process will be crucial in developing the interface to fulfill user expectations in terms of design, flow, and functionality.

Lastly, **workshops and focus groups** will be conducted to encourage collaborative conversations among Fresh Grocers workers, including IT personnel, business management, and end users. These discussions will be crucial in verifying requirements, settling disagreements, and determining development priorities. Participants will cover topics such as system scalability, data security, and integration with third-party services like Twilio and SendGrid. The training will also guarantee that all legislative, language, and accessibility aspects are handled.

Overall, these fact-finding methodologies will provide a comprehensive and trustworthy foundation for designing a user-centric, scalable, and secure digital platform that fits the operational and strategic goals of Fresh Grocers.

3.10 Selecting a suitable methodology to further analyze and review the system requirements.

To efficiently assess and review the full system requirements of the Fresh Grocers web-based platform, the Agile methodology will be the most suited way. Agile is an iterative, user-focused development technique that stresses flexibility, continuous feedback, and incremental delivery, all of which connect nicely with the complexity and growing nature of Fresh Grocers' functional and non-functional needs.

The Fresh Grocers system incorporates a wide range of functional components, including secure user registration, order placement, location-based delivery agent assignment, and customer service representative (CSR) interventions, which are interdependent and necessitate regular user engagement. With Agile, the development team will break these features down into small, manageable increments known as user stories or sprints, enabling developers to deliver working software at the conclusion of each iteration. For example, the team can prioritize a sprint focused on establishing the customer ordering interface, complete with SMS notifications and order placement features, before moving on to construct CSR functionalities or delivery agent interfaces. This guarantees that stakeholders can review and validate each module as it's produced, eliminating the danger of developing an unworkable or misaligned system.

Agile's emphasis on collaboration and stakeholder involvement is also highly relevant. Fresh Grocers' users comprise a wide spectrum, non-tech-savvy clients, CSRs, and delivery agents, each with specific expectations. Agile emphasizes regular interactions between developers and users through daily stand-ups, sprint reviews, and retrospectives. This direct channel of contact would allow developers to record comprehensive input on critical capabilities such as SMS localization in Sinhala or Tamil, order confirmation methods, or manual location updates by CSRs, enabling continual refinement and speedy adaption to customer needs.

The system's non-functional criteria, such as performance, security, scalability, and maintainability, further validate Agile's applicability. Since these features affect user happiness and long-term system viability, they cannot always be completely anticipated during original planning. Agile's iterative cycles allow for progressive optimization. For example, during early sprints, developers can incorporate basic security protocols such as HTTPS and hashed passwords. As the system scales and additional users come online,

innovations such as two-factor authentication or DDoS protection can be gradually implemented without disturbing the entire design. Similarly, Agile's support for modular development makes it easy to establish and later upgrade fundamental elements like the delivery rating system or notification engine.

Agile is also great for systems requiring frequent integration with external services like Twilio for SMS or SendGrid for email. These third-party APIs often update or change, and Agile's continuous testing and deployment (CI/CD) procedures will ensure that such integrations are monitored and modified over time. Agile also matches well with the system's portability and compatibility goals, allowing for constant browser and device testing to assure optimal performance across mobile, desktop, and various operating systems.

From a technical infrastructure standpoint, Agile encourages the use of current development tools and environments like GitHub, Docker, and cloud-hosted CI/CD pipelines. These technologies provide automated testing, rapid deployment, and real-time monitoring, all of which are crucial for maintaining the availability, security, and resilience of the Fresh Grocers platform.

Furthermore, Agile provides strong mechanisms for handling risk and change, both of which are likely in a project with such diverse customer needs and evolving market expectations. For example, if user input during pilot testing suggests a need for incorporating map-based location selection or multi-language SMS templates, Agile enables for such adjustments to be incorporated into future sprints without derailing the core timeline or structure.

In conclusion, Agile methodology offers a responsive, user-centric, and technically sound approach to system development that is particularly suited to Fresh Grocers' operating context. Its versatility, continuous improvement methodology, and heavy emphasis on cooperation and feedback will ensure that the platform not only meets but evolves with user needs, market trends, and technology advancements.

3.11 Implementing the selected methodology to further analyse and review the System Requirements

Implementing the Agile methodology to examine and revise the system requirements of Fresh Grocers gives a practical and adaptive approach to software development. Agile stresses iterative progress, frequent feedback, and cooperation, which are vital for establishing a dynamic and user-centered platform like Fresh Grocers. The system offers a range of interrelated features such as secure user registration, real-time order placement, delivery management, and performance optimization. Using Agile guarantees that these features are produced in manageable pieces, tested early, and modified based on real-world feedback from stakeholders.

Agile breaks the development process into short cycles called sprints. Each sprint focuses on a tiny, functional component of the system. For example, the first sprint can comprise implementing the user registration and authentication functionalities. This would entail allowing consumers and delivery staff to join up online and receive their access credentials via email. By finishing and assessing this module early, developers may guarantee security, usability, and correct email integration before moving on.

Subsequent sprints may focus on client order placement, a vital component of the Fresh Grocers platform. This functionality lets clients to explore products, choose things, and confirm their order. Once the order is confirmed, the system must send an SMS notice containing the designated delivery agent's contact details and anticipated delivery time. Agile helps developers to construct and test these processes iteratively, ensuring each component functions perfectly before additional features are added.

Another essential aspect is allowing customer service agents (CSRs) to place orders on behalf of clients. Using Agile, the development team can first focus on enabling CSRs to input phone numbers and locations, and then progressively implement advanced capabilities like automatic SMS confirmation and agent assignment. Feedback from actual CSRs during testing can help build a more intuitive interface and better workflows.

Delivery agent assignment is a complex process that depends on location data. Agile helps testing and improving the logic that detects nearby delivery agents and shows them as options for customers or CSRs. By assessing real usage scenarios in short cycles, developers can tweak algorithms and interface elements to ensure efficient agent assignment.

Location management, which permits both manual and CSR-assisted location updates, can also benefit from Agile's flexible framework. Early versions might contain simple forms for location entry, but later iterations could add automatic location tracking or connections with GPS equipment.

Agile is as successful at integrating non-functional requirements. Features like as performance, scalability, and security can be introduced with each sprint. For instance, developers can begin with password hashing and secure HTTPS protocols and then grow to incorporate role-based access control, input validation, and security against cyber-attacks. These aspects are tested often, decreasing the possibility of vulnerabilities upon launch.

Usability is a key non-functional criteria that Agile supports well. Continuous stakeholder engagement during sprint reviews allows developers to get feedback from customers, CSRs, and delivery personnel. This ensures that the interface remains intuitive and that enhancements are based on actual user needs, not assumptions.

Agile also increases maintainability and modular development. Each sprint offers a complete, testable module such as the SMS notification system or the delivery rating feature which can be tweaked independently without impacting other components. This modular architecture makes it easy to incorporate future enhancements, such new rating filters or expanded reporting capabilities.

Portability and compatibility are vital for Fresh Grocers, which must perform across devices and browsers. Agile improves cross-platform development by promoting regular testing on many operating systems and screen sizes. External API connectors, such as Twilio for SMS or SendGrid for emails, can be gradually included and tested during multiple sprints.

The technique also assists infrastructure planning and system requirements. Developers can construct staging environments that mimic the real setup, enabling safer testing and deployment. Tools like Git for version control, Docker for containerization, and Jenkins for continuous integration ensure that the system remains stable and scalable as new features are added.

Backup and recovery processes can also be incrementally created and tested. Automated backup schedules, data retention policies, and disaster recovery methods can be incorporated in the final stages of development, ensuring the system is well-protected before full deployment.

In summary, Agile provides a disciplined, iterative way to examining and revising the functional and non-functional requirements of the Fresh Grocers system. It emphasizes

early delivery, rapid feedback, and continual improvement, which guarantees the final product is reliable, user-friendly, secure, and scalable. Through Agile, the development team can construct a resilient platform that adapts with customer needs and supports the long-term aims of the grocery delivery service.

3.12 Assessing the effectiveness of the methodology used in providing a solution for a given business context.

The evaluation of the analysis and review against the proposed solution for Fresh Grocers' web-based grocery ordering platform indicates a solid alignment between the system requirements and stakeholder needs, while also validating the solution's viability for implementation. The implementation of Agile approach has been essential in ensuring that the functional and non-functional requirements are not only properly stated but also flexible to evolving user feedback and operational realities. This synergy creates a foundation for producing a successful, efficient, and user-centric system.

From the onset, Agile's iterative methodology facilitates the delivery of essential functionalities in manageable increments, which corresponds well with Fresh Grocers' complicated needs such as secure user registration, real-time order placement, delivery agent assignment, and customer feedback systems. By breaking down development into sprints concentrating on distinct parts, the system can be regularly tested and refined to meet both customer and internal stakeholder requirements. This ensures early detection and resolution of issues relating to usability, security, or integration, addressing potential concerns before they worsen. For example, the staged development of customer registration and authentication, followed by order processing and SMS alerts, provides rigorous evaluation of crucial procedures central to user trust and operational efficiency.

The proposed solution, aimed to replace the present manual ordering system, clearly solves the primary pain points such as delays, miscommunication, and restricted transparency. The new platform's emphasis on automation, user-friendly interfaces, and real-time communication directly fulfils stakeholders' expectations for a speedier, more dependable service. The option for clients to place orders online and receive fast SMS confirmations minimizes dependency on phone-based orders and customer support interventions, therefore boosting service speed and accuracy. Moreover, the integration of

location-based delivery agent assignment reduces logistics and operational costs, proving that the system is tailored not only to suit user expectations but also to enhance company efficiency, a crucial stakeholder concern.

Non-functional needs, including security, performance, scalability, and maintainability, are appropriately handled in both the analysis and the suggested solution. The use of encrypted user data, secure login methods, and role-based access restrictions assures compliance with modern security requirements, protecting both consumer and company information. Agile's emphasis on continuous testing provides robust performance under simultaneous user activity and prepares the system to scale across expanding service areas. The modular architecture enables for continuing maintenance and feature upgrades without compromising general functionality, coinciding with long-term company sustainability.

Feasibility is adequately supported through detailed hardware and software specifications, infrastructure planning, and a defined testing and deployment strategy. The proposed server specifications, software stacks, API connectors, and development tools mirror industry best practices, making the technological implementation viable within the organizational context of Fresh Grocers. The incorporation of backup and recovery plans, disaster recovery processes, and logging methods further enhances system reliability and auditability, addressing operational risks proactively.

To ensure the recommendations are actionable, the following elements are critical: first, the development team must maintain constant communication with stakeholders during each Agile sprint to include real-time input and alter priorities as needed. This technique guarantees that evolving user needs and any emerging technical issues are addressed swiftly. Second, precise documentation of functional and non-functional requirements must be maintained and updated alongside the system's ongoing expansion to minimize scope creep and misalignment. Third, early investment in infrastructure, notably scalable hosting environments and secure network configurations, will underlie system stability while user load increases.

In conclusion, the analysis and assessment establish that the proposed web-based ordering platform meets the Fresh Grocers' stakeholder needs thoroughly while being technically and operationally practical. Agile technique ensures flexibility, reactivity, and quality throughout the development lifecycle, so enabling the establishment of a reliable, user-friendly, and secure system. By following the actionable advice of constant stakeholder

involvement, rigorous testing, and strong infrastructure planning, Fresh Grocers is well positioned to effectively execute this solution and create a competitive advantage in Sri Lanka's expanding digital grocery sector.

Activity 04

4.1 User and System Requirements

The proposed grocery ordering and delivery system is aimed to modernize and automate the process of grocery shopping by enabling customers, delivery workers, and customer service representatives (CSRs) to engage through a streamlined web-based interface. This system is created with an emphasis on usability, performance, security, and scalability, ensuring it fulfils both user expectations and technical standards.

From a user perspective, the system is expected to deliver frictionless registration and authentication operations. Customers and delivery agents should be able to register online, following which the system sends an email providing a unique username and password for login. Once registered, users must be able to safely log into the system using their credentials. For clients, the ability to browse and select grocery items online is a vital demand. Once an order is placed, the system should quickly send an SMS confirmation providing critical facts such as the assigned delivery agent's contact information and the projected delivery time.

Moreover, the system must permit customer support personnel to place orders on behalf of consumers without needing to log in as them. In such circumstances, the CSR should input the customer's phone number and location, and the system must still send a confirmation SMS to the provided number, guaranteeing transparency and communication. The process of appointing delivery agents is also a critical user requirement. The system should intelligently detect and show the closest available agents depending on the order's location and allow either the customer or CSR to choose from the list.

In addition to order handling, effective location management is important. Delivery agents should be able to manually update their current location, and CSRs must also be permitted to submit location details on their behalf. Customers should be asked to provide their location either manually or by assistance from the CSR while placing orders. After each delivery, the system should prompt clients to rate their delivery experience. These

ratings must be recorded and explicitly linked to the delivery agent's profile to help evaluate performance and service quality over time.

To achieve these objectives, the system must be supported by a robust technical infrastructure. On the server side, the platform should run on high-performance hardware with CPUs like Intel Xeon or AMD Ryzen Threadripper, 8–32 GB of RAM for handling concurrent users, and 1 TB of SSD storage to provide rapid access and dependability. It should also maintain a high-speed internet connection of at least 1 Gbps and deploy a reliable backup mechanism, either external or cloud-based. Client devices can range from PCs and laptops to smartphones and tablets, and they should meet minimal specs, such as dual-core processors and sufficient RAM and storage, to interact with the web-based platform efficiently.

The server software stack should comprise current and trustworthy components such as Ubuntu Server or Windows Server for the operating system, Apache or Nginx as the web server, and a relational or NoSQL database like MySQL, PostgreSQL, or MongoDB. Backend development can be carried out using Laravel (PHP), Node.js, or Django (Python), while the frontend should be constructed using HTML5, CSS3, and JavaScript frameworks such as React or Angular to ensure a responsive and user-friendly interface. To provide timely communication, the system needs interact with APIs such as Twilio for SMS and SendGrid or Mailgun for email services.

Security is crucial in ensuring the trust and safety of all users. The platform must implement secure authentication procedures such as password hashing (bcrypt) and possibly, two-factor authentication. All data transmissions should be secured using HTTPS with TLS 1.2 or above, and sensitive data must be encrypted to prevent breaches. Role-based access control is important to divide the capabilities accessible to customers, delivery staff, and administrators. Additionally, a comprehensive firewall setup and anti-DDoS protection methods should be in place to defend the platform from cyber threats.

Scalability and performance are also critical concerns. The system must support concurrent access by numerous users making orders or altering locations, and it must be architected to manage expansion throughout cities and suburban areas. High availability must be maintained, especially during peak supermarket hours, with little downtime. To achieve this, automated testing and deployment pipelines should be developed using tools like Jenkins or GitHub Actions, and monitoring tools such as New Relic or Prometheus should be set to track system health and performance in real time.

Usability is another key consideration. The interface must be intuitive, allowing even non-tech-savvy users to simply traverse the site. The CSR interface must support fast and efficient manual data entry to help clients efficiently. The system should be browser-independent and mobile-responsive to accommodate a wide range of devices and user preferences.

Maintenance and extensibility are assisted by using a modular software architecture, allowing new features, like updated rating algorithms or alternative notification methods to be added without interrupting existing services. The system should also support localization of communication, with SMS and email templates available in Sinhala, Tamil, and English to accommodate regional users. Furthermore, the notification systems should be configurable and scalable, leveraging third-party gateways such as Twilio.

For transparency and accountability, the platform should preserve extensive logs of user activity, including order placements, location updates, CSR interventions, and rating submissions. These logs must be accessible to administrators for auditing and troubleshooting purposes.

Finally, backup and recovery methods must be in place to preserve data integrity. Regular backups; daily for databases and weekly for full-site data should be automated, with a documented disaster recovery strategy such as failover servers or cloud-based mirrors. A clear data retention policy should also be created, with customer and order data retained for six months to one year, assuring compliance and operational preparedness.

In conclusion, the grocery ordering and delivery system must smoothly blend user-centric features with a stable and secure technical base. The platform's success rests on its capacity to deliver a dependable, scalable, and intuitive experience while being safe, maintained, and ready for future development.

4.2 System Designs

4.2.1 System Architecture

(Zoom for a clearer view)

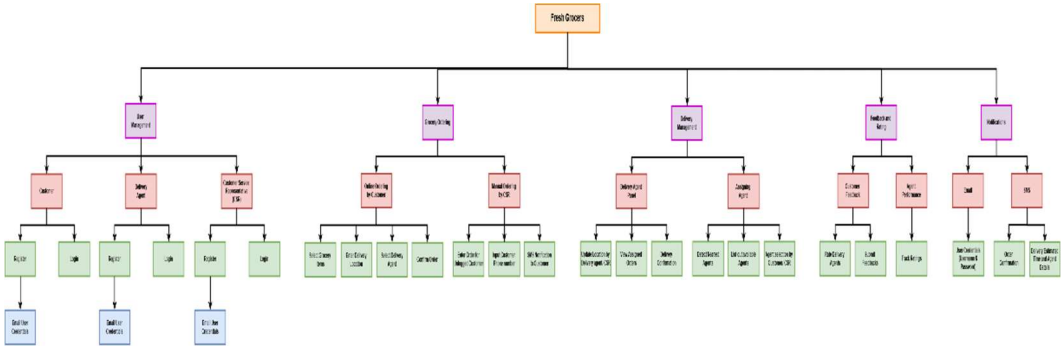


Figure 9: System Architecture Design

4.2.2 Data Flow Diagram (Level 0)

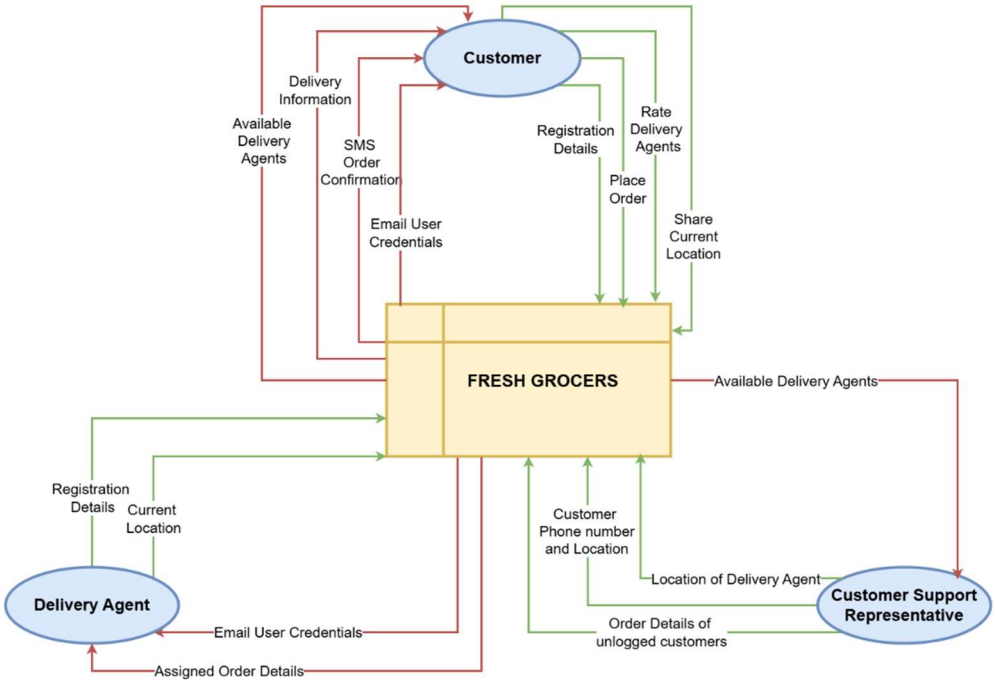


Figure 10: Data Flow Diagram (Level 0)

4.2.3 Data Flow Diagram (Level 01)

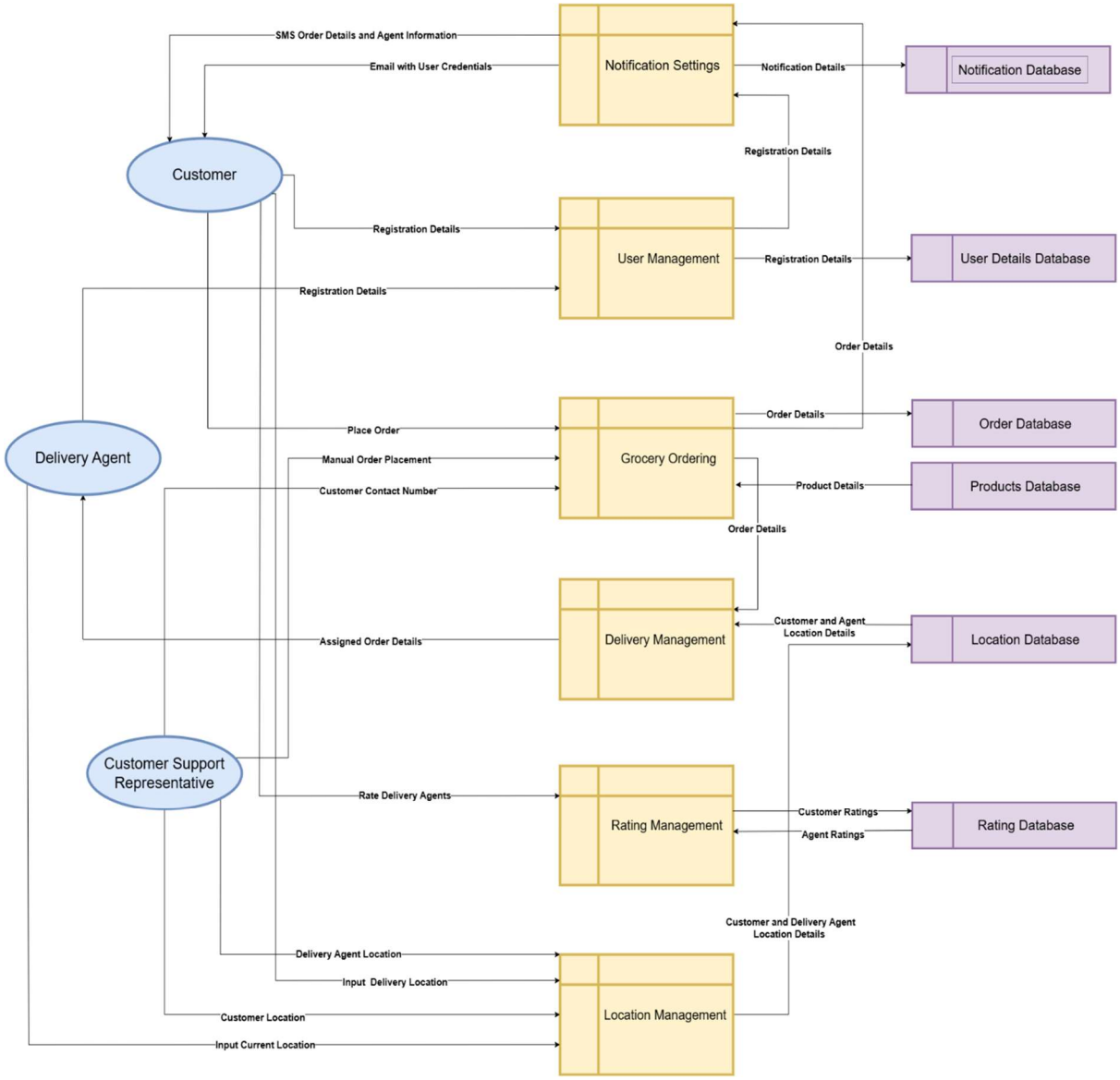


Figure 11: Data Flow Diagram (Level 01)

4.2.4 Use Case Diagram

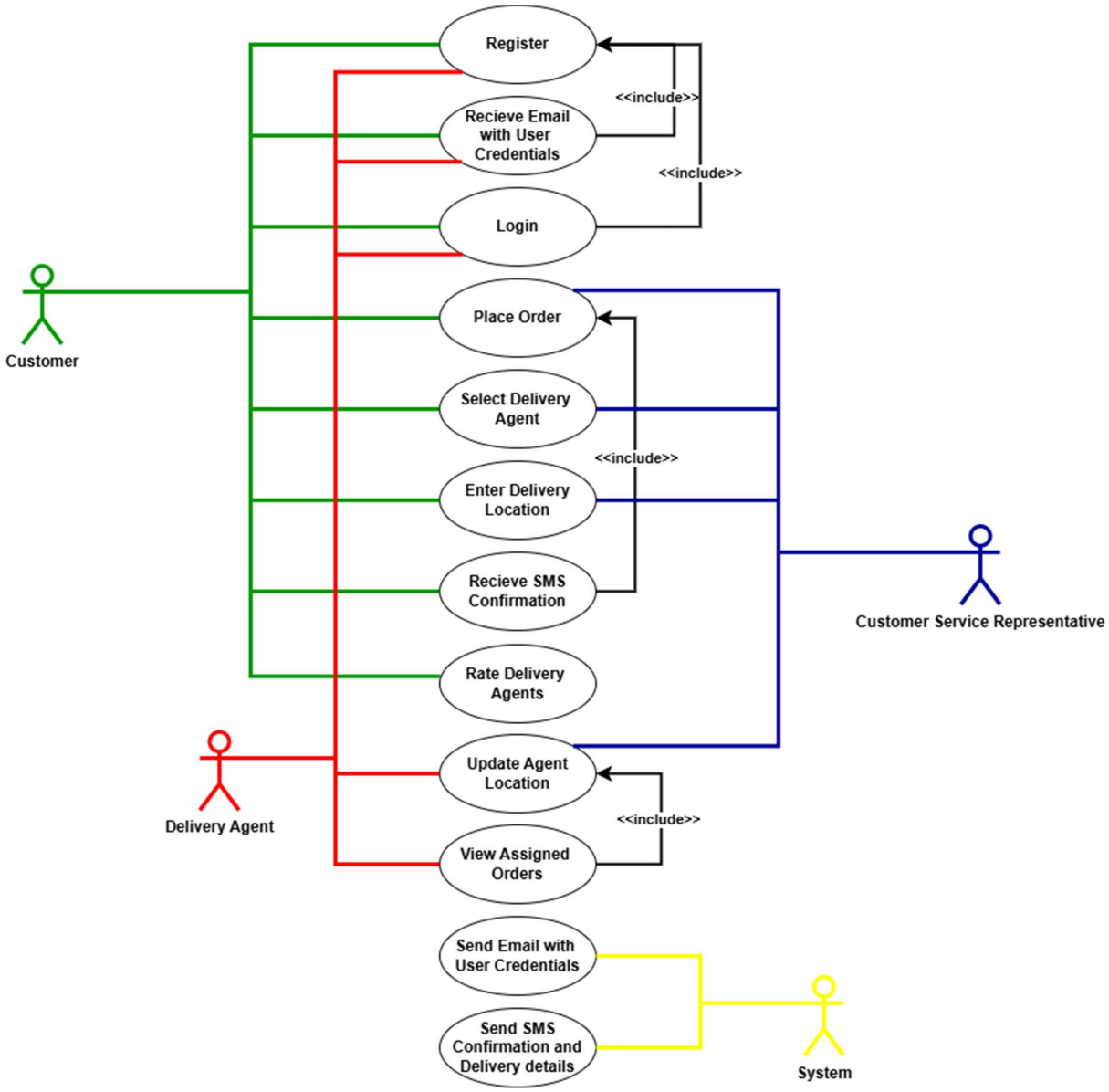


Figure 12: Use Case Diagram

4.2.5 User Interface Designs

Registration Form

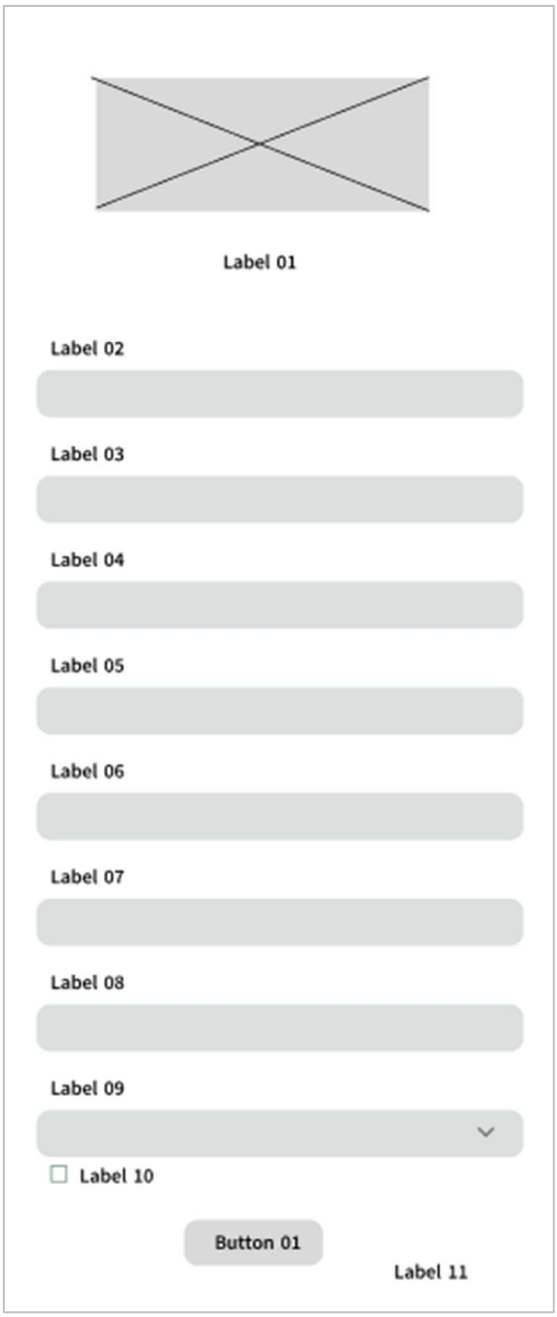


Figure 13: Wireframe: User Registration.

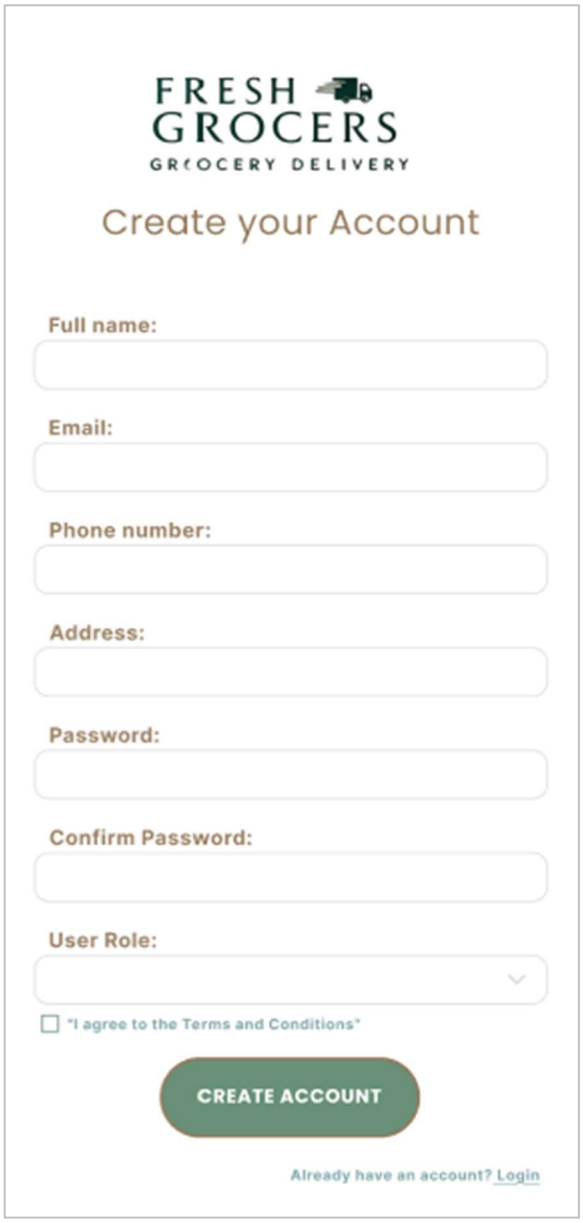


Figure 14: Prototype: Registration

Login

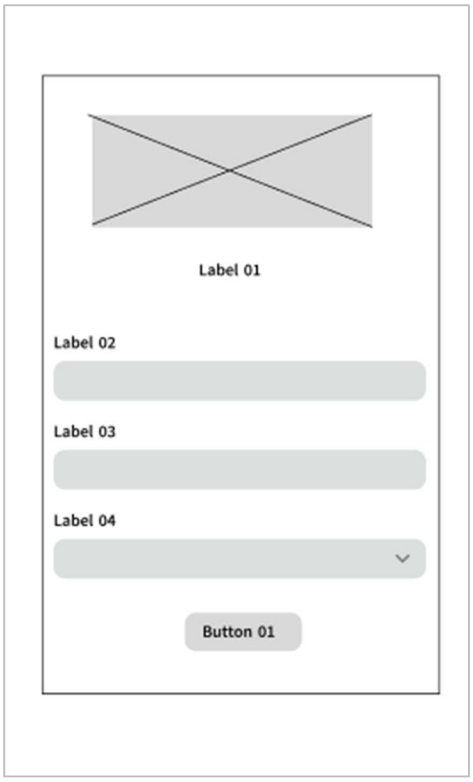


Figure 15: Wireframe: Login

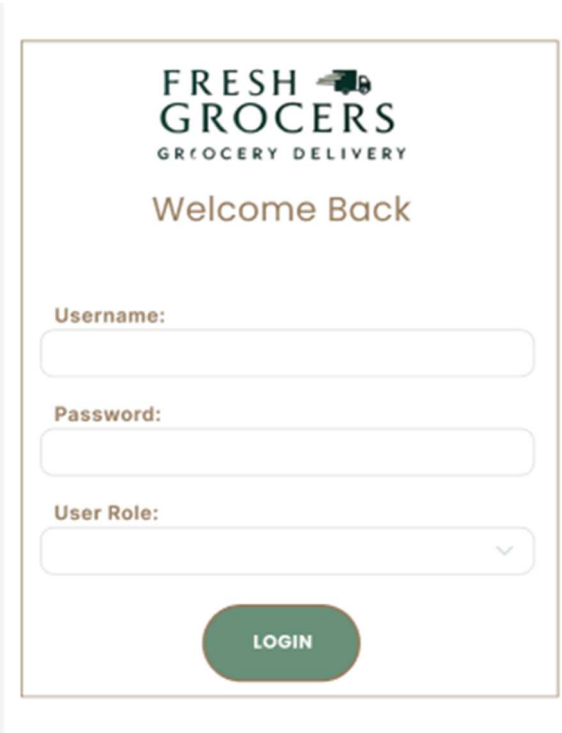


Figure 16: Prototype: Login

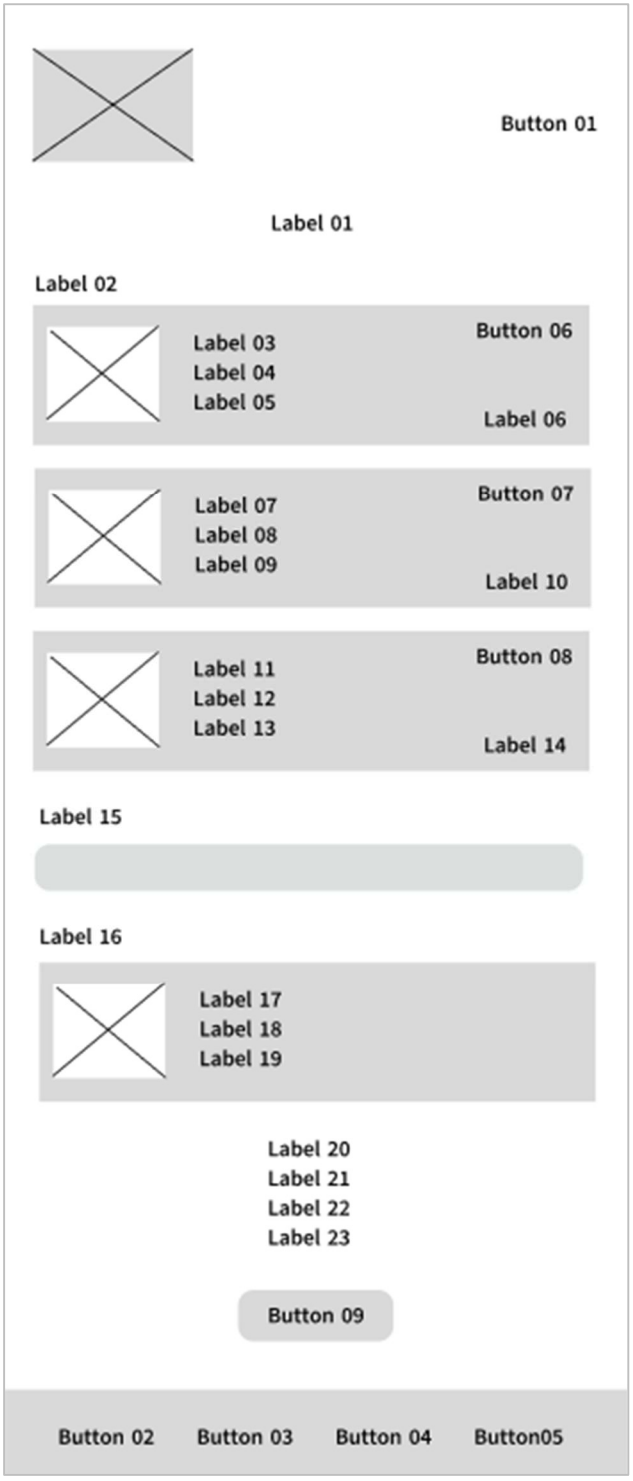


Figure 17: Wireframe: Customer Order Placing.

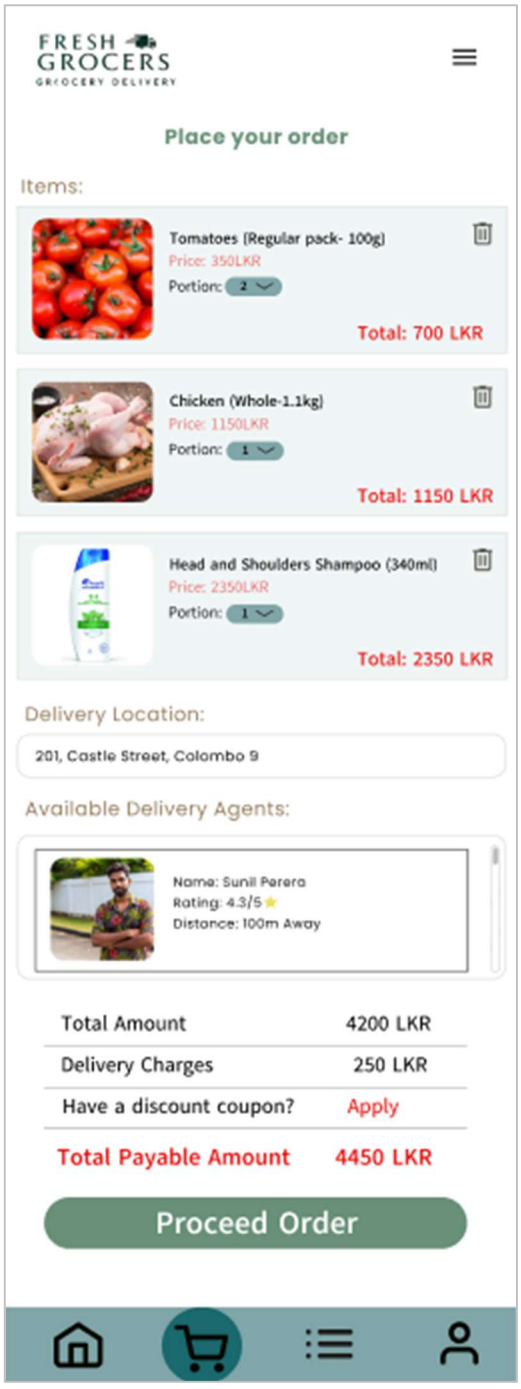


Figure 18: Prototype: Customer placing order

Customer Service Representative placing order

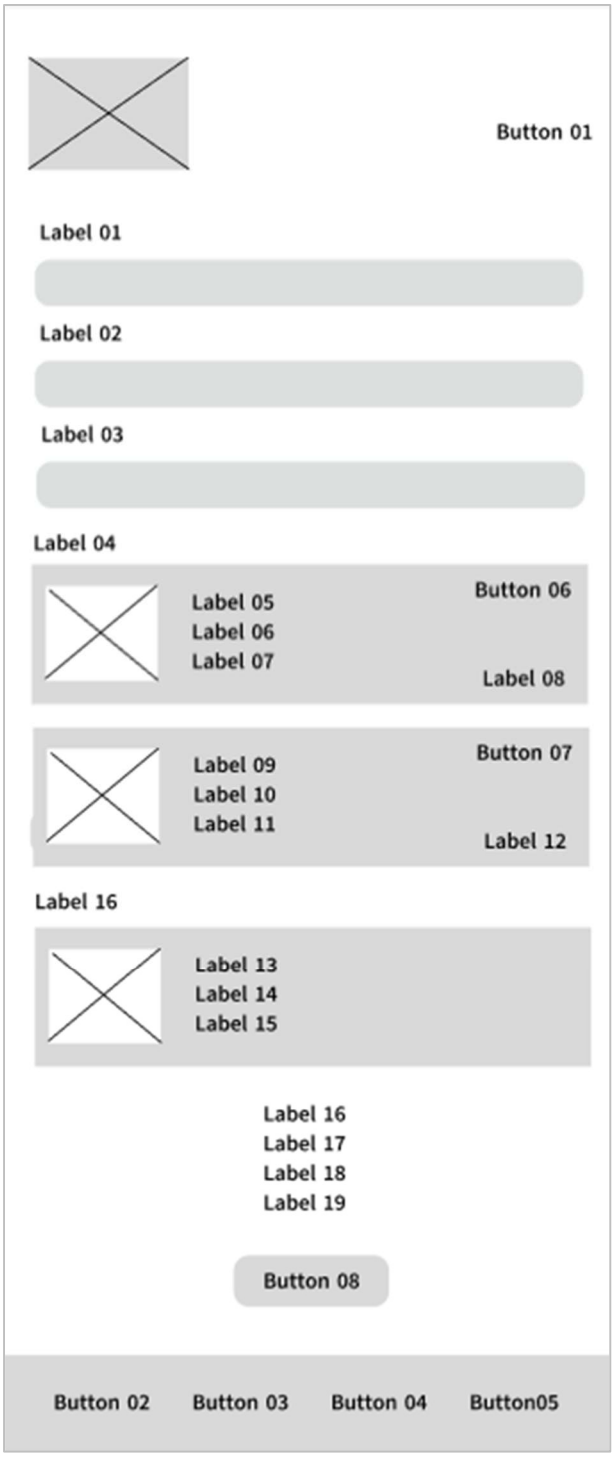


Figure 19: Wireframe: CSR placing order.

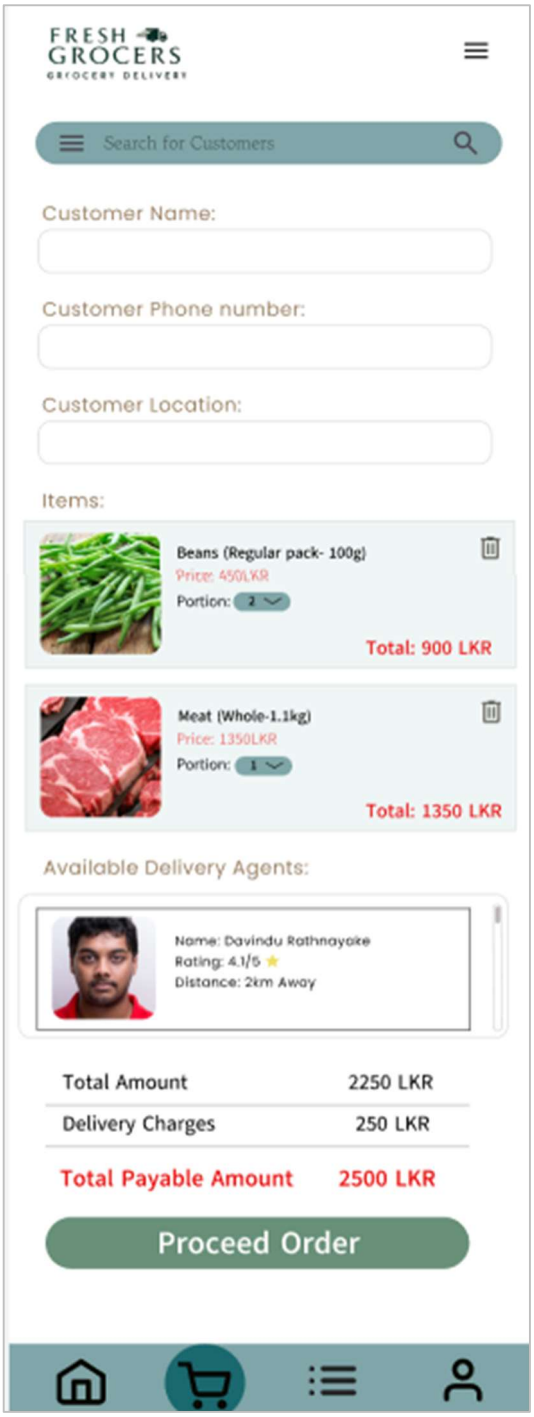


Figure 20: Prototype: CSR placing order

Delivery Agent Dashboard

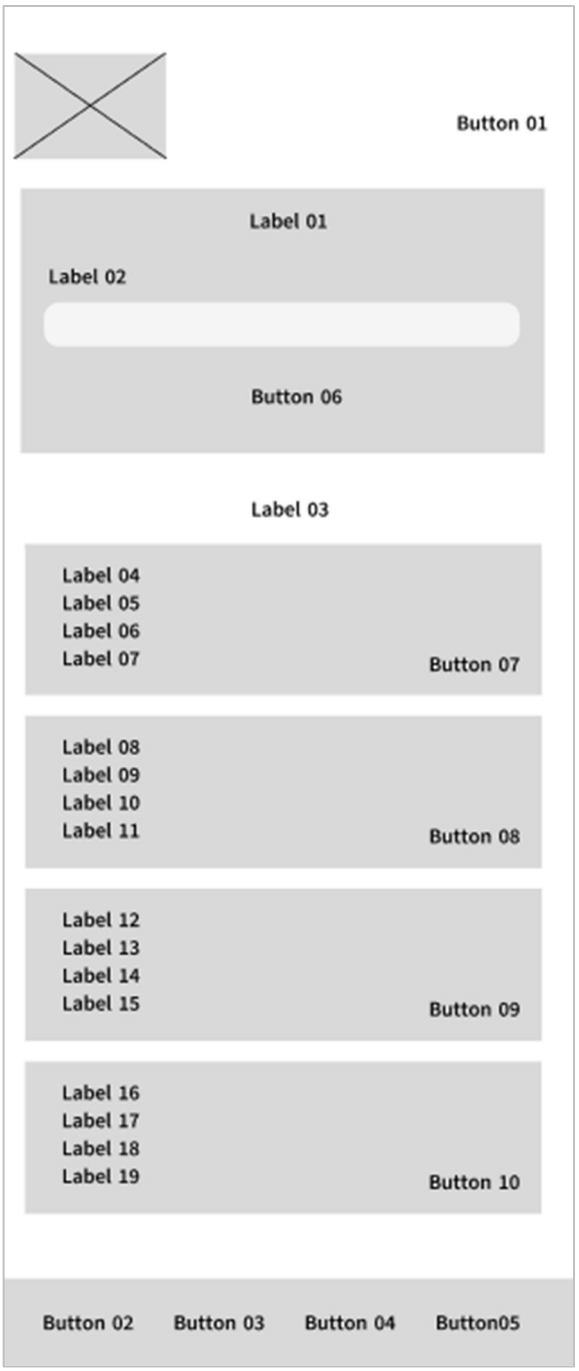


Figure 21: Wireframe: Delivery Agent Dashboard.

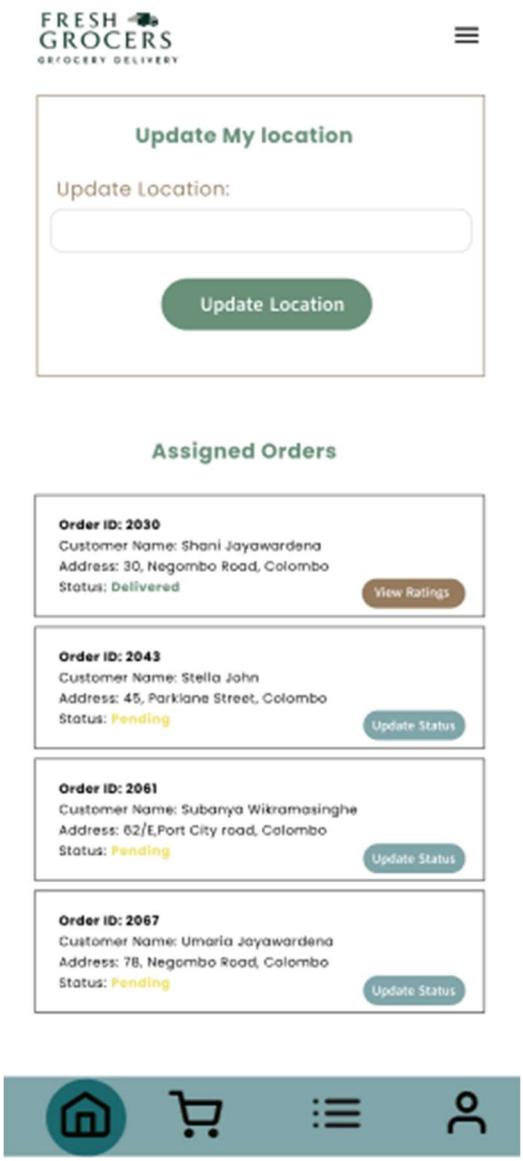


Figure 22: Prototype: Delivery Agent dashboard

Customer Rating Delivery Agent Page

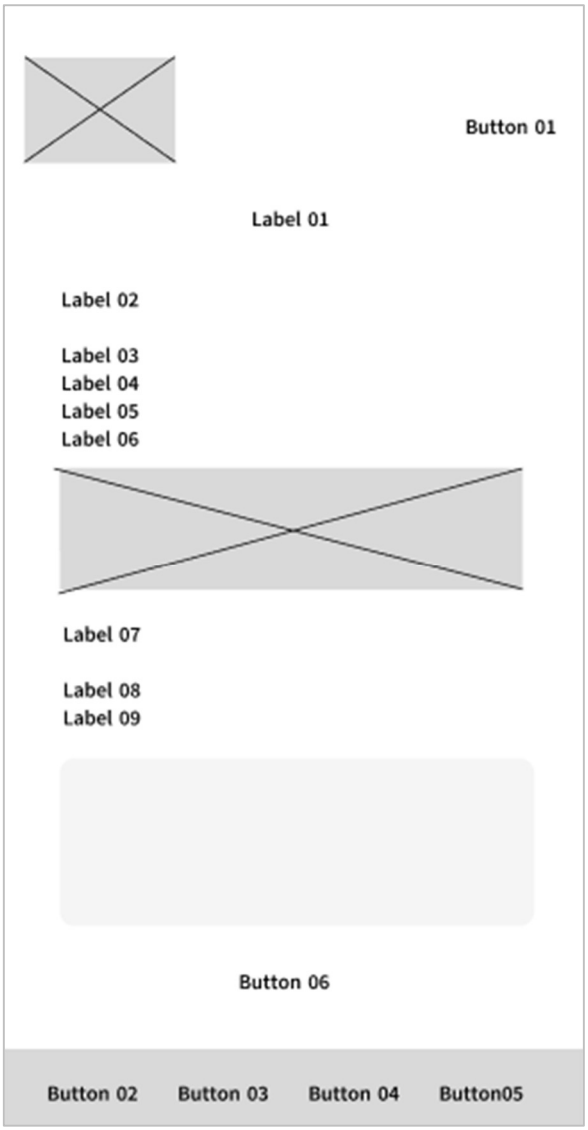


Figure 23: Wireframe: Customer Rating Delivery Agent.

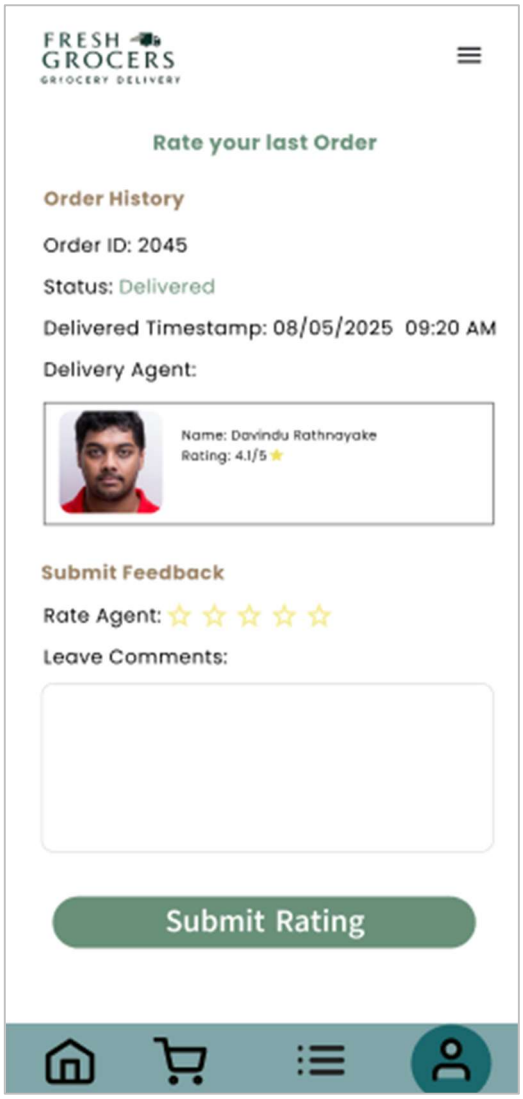


Figure 24: Prototype: Customer rating Delivery Agent.

4.2.6 Database Design

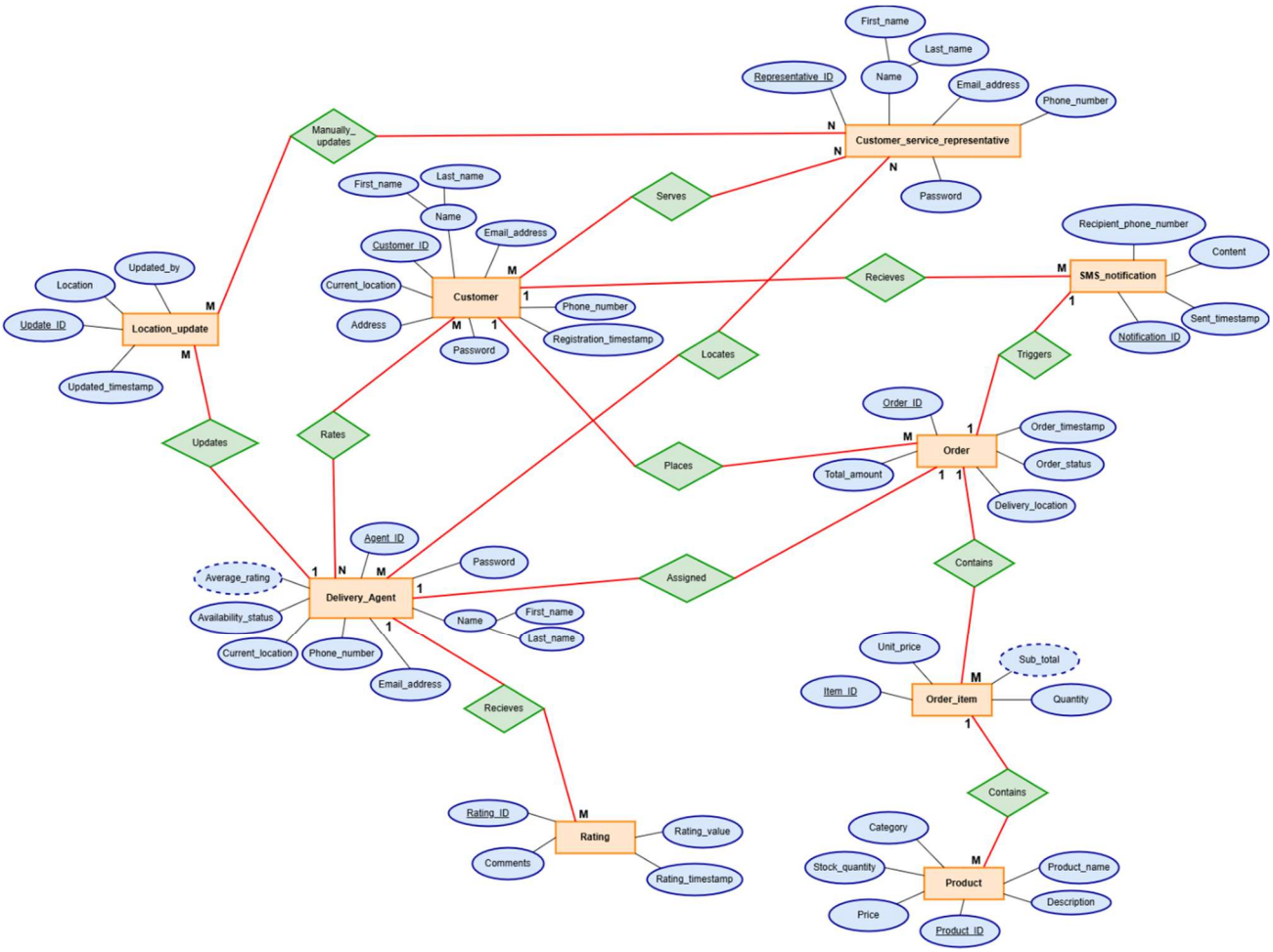


Figure 25: Database Design.

4.2.7 Programme Designs

User Registration

Flowchart:

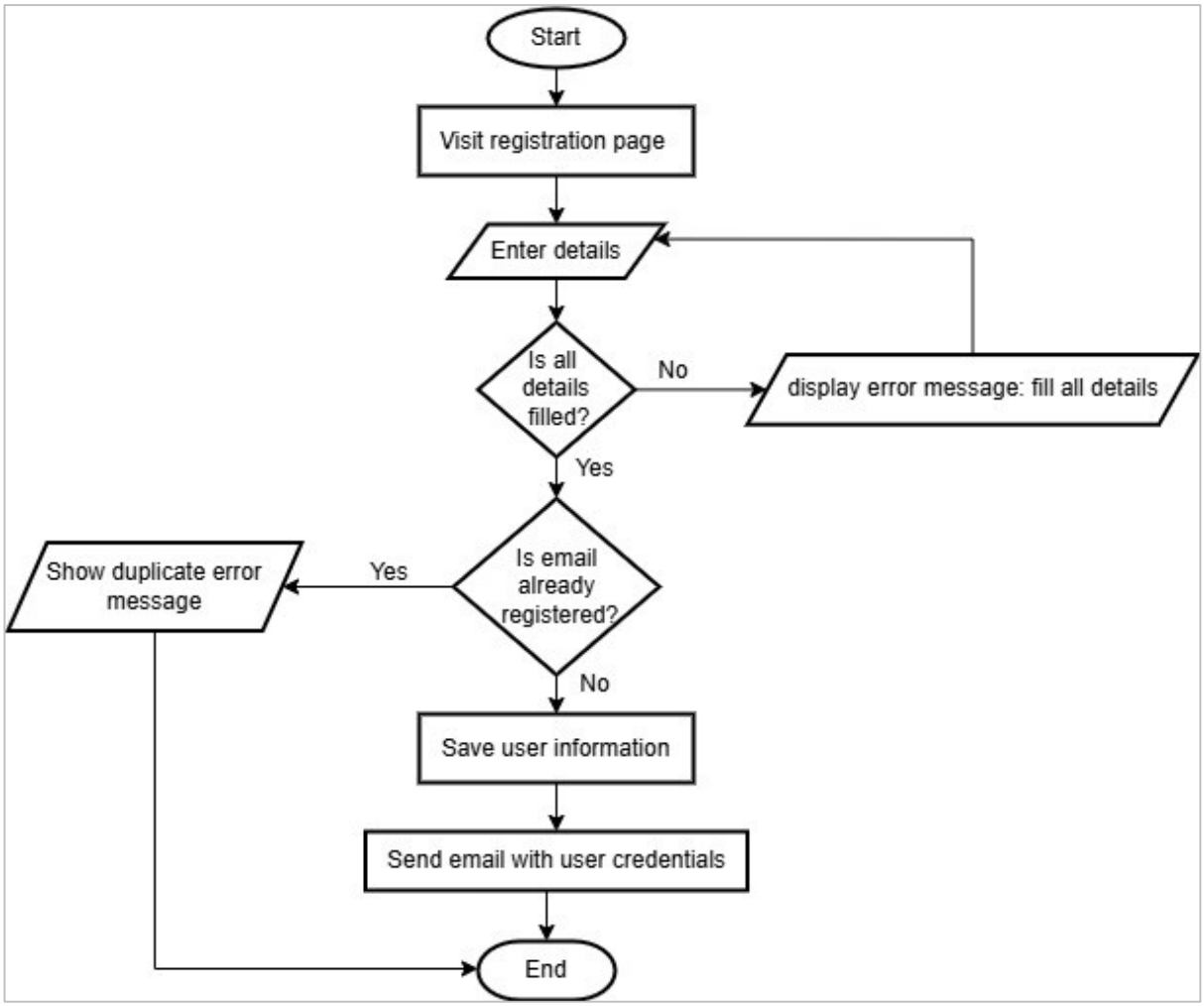


Figure 26: Flowchart: Registration.

Pesudocode:

```
start
Visit registration page
Input user details
        if any detail missing then
                 Display message "Fill all details"
        else
                if email already registered then
                        Display message "Duplicate email error"
                else
                        Save user information
                        Send email with user credentials
                end if
        end if
stop
```

Figure 27: Pseucocode: Registration

Customer Ordering

Flowchart:

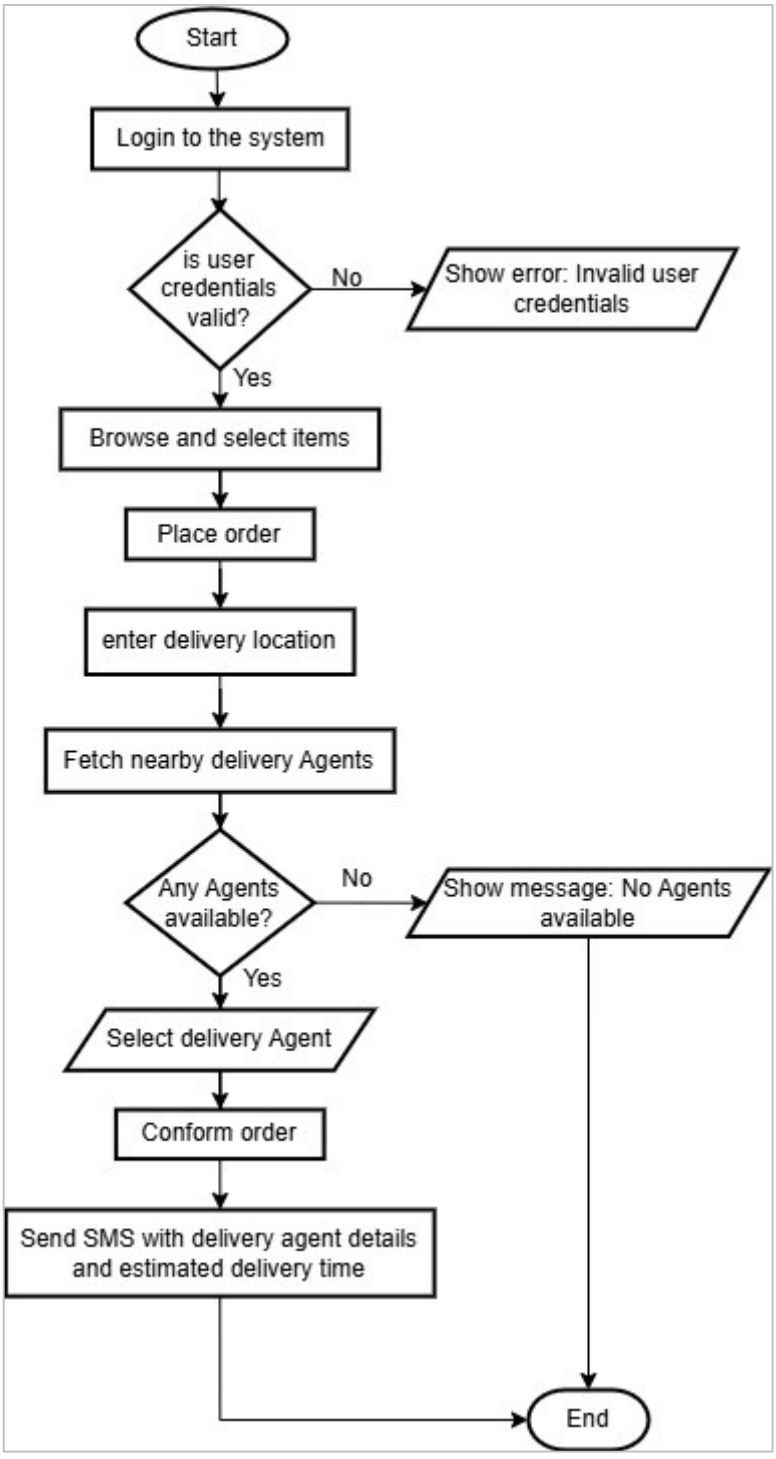


Figure 28: Flowchart Customer Ordering.

Pseudocode

```
start
Login to the system
        if user credentials are not valid then
                Display message “Invalid user credentials”
        else
                Browse and select items
                Place order
                Enter delivery location
                Fetch nearby delivery agents
                if no agents are available then
                        Display message “No Agents available”
                else
                        Select delivery agent
                        Confirm order
                        Send SMS with delivery agent details and estimated delivery time
                end if
        end if
stop
```

Figure 29: Pseudocode: Customer Ordering

Customer Service Representative Order placement

Flowchart:

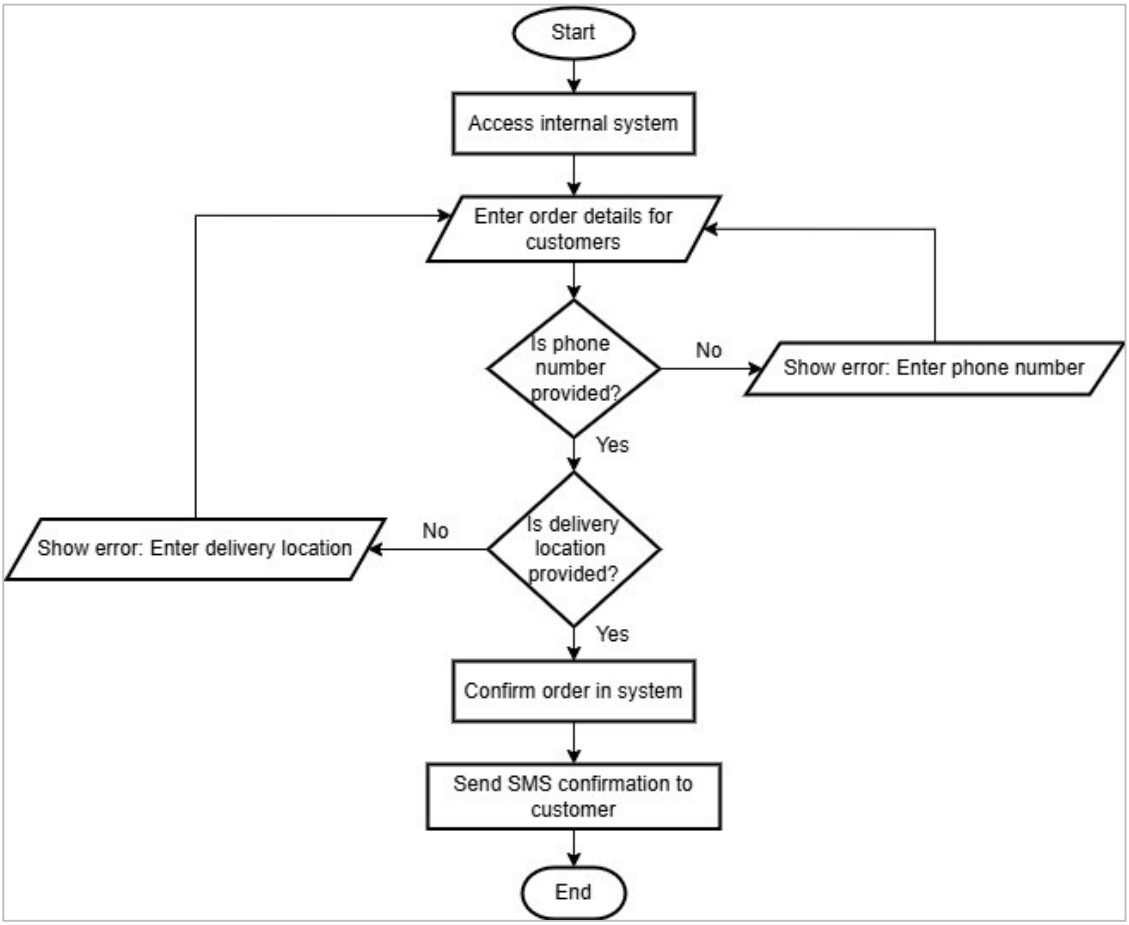


Figure 30: Flowchart: CSR Ordering

Pseudocode:

```
start
Access internal system
Input customer order details
        if phone number is not provided then
                Display message "Enter phone number"
        else
                if delivery location is not provided then
                        Display message "Enter delivery location"
                else
                        Confirm order in system
                        Send SMS confirmation to customer
                end if
        end if
stop
```

Figure 31: Pseudocode: CSR Ordering

Delivery Management

Flowchart:

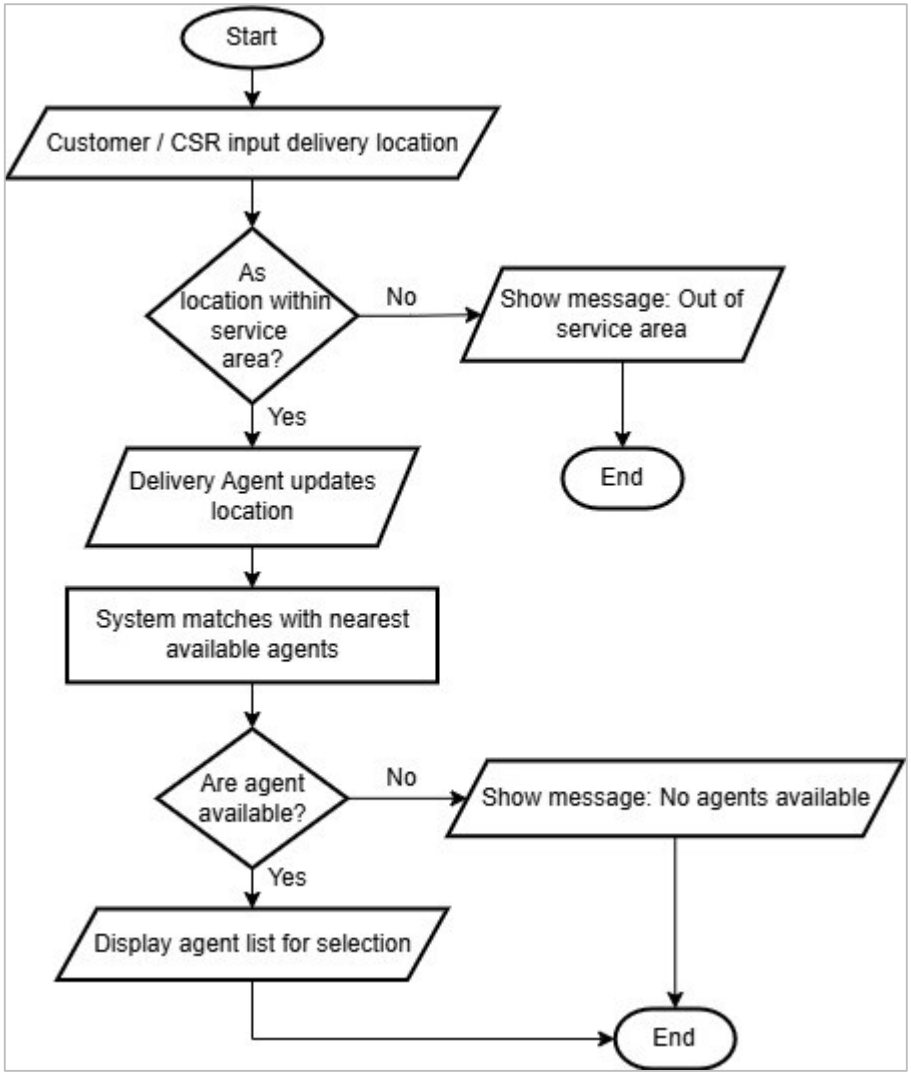


Figure 32: Flowchart: Delivery Management.

Pseudocode

```
Start
Input delivery location
        if location is not within service area then
                Display message “Out of service area”
        else
                Delivery agent updates location
                System searches for nearest available agents
                if no agent available then
                        Display message “No agents available”
                else
                        Display agent list for selection
                end if
        end if
Stop
```

Figure 33: Pseudocode: Delivery Management.

Delivery Agent Update Location

Flowchart:

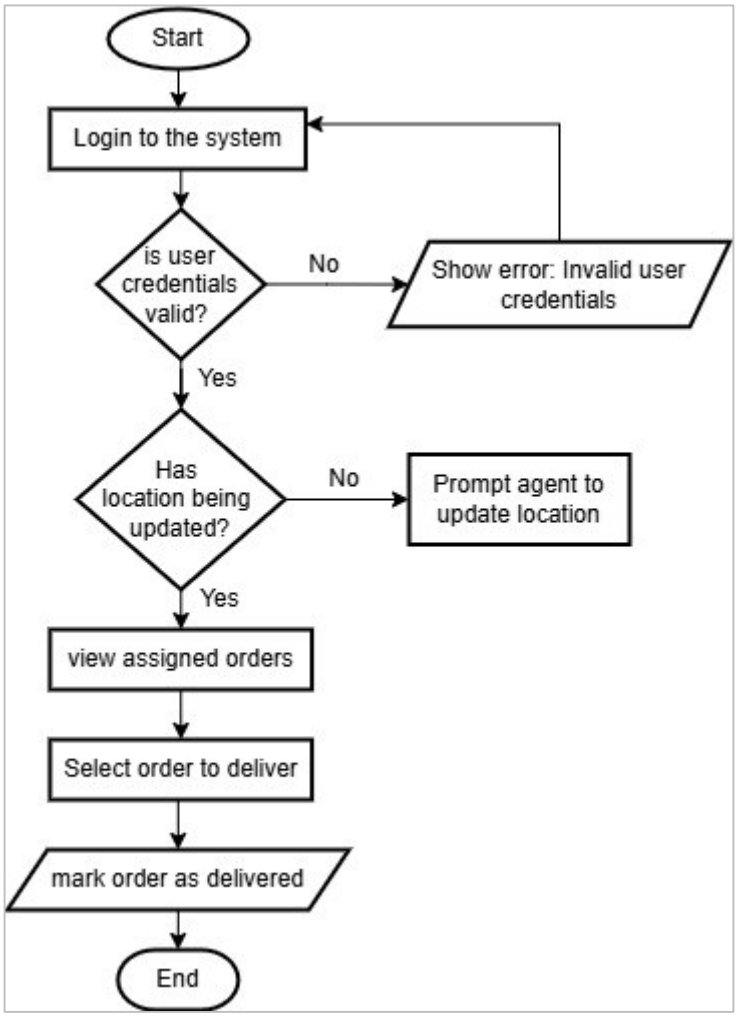


Figure 34: Flowchart: Delivery Agent Update Location.

Pseudocode:

```
Start
Login to the system
        if user credentials are not valid then
                Display error “Invalid user credentials”
        else
                if location has not been updated then
                        Prompt agent to update location
                else
                        Display assigned orders
                        Select order to deliver
                        Mark order as delivered
                end if
        end if
Stop
```

Figure 35: Pseudocode: Delivery Agent Update Location.

4.3 Review the design and compare it against the user and system requirements.

The system design for the Fresh Grocers web-based ordering and delivery platform exhibits a deliberate and well-structured approach that aligns closely with the established user and system requirements. Upon reviewing the uploaded design elements such as architecture diagrams, data flows, user interfaces, and program logic, it becomes evident that the solution is largely effective in meeting the functional expectations of customers, delivery personnel, and customer service representatives (CSRs), while also addressing several key technical considerations.

From a user standpoint, one of the most significant functionalities is registration and login. The system features specific user interface screens for registration and login, guaranteeing that both customers and delivery staff can quickly create accounts and use the system securely. The process of sending a confirmation email with unique credentials is part of the registration flow, aligning with the requirement for secure user authentication. The login interface is likewise clean and user-friendly, reflecting the emphasis on accessibility for a wide variety of users, including those with less technical skills.

The client order placement process is another area where the design matches the needs effectively. The user interface allows users to explore available groceries, add items to their cart, and complete their order. After confirmation, the system is meant to send an SMS to the customer including the contact details of the assigned delivery agent and the projected delivery time. This feature promotes transparency and meets the requirement for real-time communication and service tracking.

The needs of CSRs are also properly handled in the design. A specialized interface allows CSRs to conduct orders on behalf of consumers, which is vital for helping users who prefer phone-based assistance. The form provides fields for entering the customer's phone number and location, ensuring that all relevant information is obtained without requiring CSRs to impersonate users. The system also supports sending SMS confirmations in these instances, keeping communication standards across different sorts of order placements.

Delivery agent administration is enabled by a dashboard that enables agents to check their allocated orders, update their status, and manually input or update their current

location. This satisfies the location management needs and enables flexibility in regions where automatic GPS tracking may not be available. Additionally, CSRs can change delivery agent positions when necessary, ensuring that the system stays accurate and responsive even with manual input.

Customer feedback is facilitated through a dedicated portal that allows users to rate their delivery experience. These ratings are maintained within the system and connected to each delivery agent's profile. This functionality directly facilitates performance monitoring and continual service enhancement, as required.

The system architecture and data flow diagrams depict a modular and logical framework that handles user interaction, order processing, and data management in a smooth manner. Each user role; customer, delivery agent, and CSR, interacts with the system through discrete, role-specific interfaces that are both intuitive and functionally rich. The backend procedures ensure that operations like registration, ordering, and delivery assignment are done efficiently, with clear data flow between components.

The database design represents the essential entities necessary for the system, such as users, orders, delivery agents, ratings, and locations. Relationships between these entities are appropriately mapped, ensuring that data retrieval and storage are optimized for performance and integrity. This framework promotes maintainability and future scalability, fitting well with the system requirement for modular development and extensibility.

The program designs, including flowcharts and pseudocode, indicate a comprehensive understanding of the procedures involved in registration, ordering, CSR engagement, and delivery management. The logic flows are sequential and practical, which allows straightforward implementation and debugging. However, some areas; particularly those connected to security; would benefit from more detailed elaboration. For instance, although password-based authentication is present, the pseudocode does not visibly reference encryption technologies such as password hashing or secure transmission protocols like HTTPS.

While the design exhibits great alignment with usability, reliability, and functionality requirements, certain system-level demands including performance optimization, backup and recovery mechanisms, and automated testing strategies are not properly addressed. Features like continuous integration/deployment (CI/CD), daily data backups, and system monitoring, although stated as necessary in the requirements are not reflected in the current architecture or development plans. These factors should be included

into future design iterations to guarantee the platform is not just useful but also durable and resilient.

Additionally, the necessity for location-based delivery agent assignment is acknowledged in the design, but the underlying method or logic that determines the "nearest" agent is not clearly described. Implementing and evaluating such logic is vital for operational efficiency and customer satisfaction, especially in a logistics-heavy system.

The multilingual support and notification customization, which were noted in the criteria, are not visibly shown in the existing UI or backend logic. To effectively serve varied user bases, the system should include SMS and email templates that may be localized in Sinhala or Tamil and allow admin control of communications formats. Integration with third-party gateways like Twilio and SendGrid is assumed but should be explicitly shown in the architecture or implementation plan.

In summary, the system design for Fresh Grocers delivers a well-thought-out solution that fulfils the primary user and system requirements with a high level of completeness. The user interfaces are intuitive and role-appropriate, the data structures are sound, and the program logic is logically sequenced and easy to understand. However, enhancements are needed in areas such as backend security implementations, infrastructure readiness, and rollout planning. With these enhancements, the system would not only meet but surpass expectations, delivering a secure, scalable, and user-centric digital grocery ordering experience.

4.4 Evaluating the methodology used to develop the system design and determine whether it was appropriate for the project.

The implementation of the Agile methodology for building the Fresh Grocers system design was very appropriate and well-suited for the nature and scope of the project. Agile, as a flexible and iterative approach to software development, emphasizes on continuous communication, early delivery of functional components, and the capacity to adapt to changing needs. Given that the Fresh Grocers project involved multiple user roles; customers, delivery agents, and customer service representatives, and required the integration of several dynamic and interdependent system features, Agile provided the ideal framework to manage complexity and ensure user-focused outcomes.

One of the main characteristics of Agile is its emphasis on incremental development through sprints, which enabled the development team to separate the system into digestible components such as user registration, order placing, delivery assignment, and feedback collecting. By focusing on one feature set at a time, the team could ensure each function was completely built, tested, and refined based on stakeholder input before continuing. This method considerably decreased the risk of rework and allowed for faster detection and resolution of possible usability or performance concerns.

Agile's principle of continuous stakeholder engagement proved particularly effective in this setting. Since the project involves a wide user base, ranging from tech-savvy online buyers to less experienced customers dependent on CSR assistance, the iterative development process allowed for frequent reviews and feedback sessions. For example, early prototypes of the customer ordering interface and CSR dashboard could be displayed to actual users, and input was swiftly included into following development cycles. This meant that the final design was not only practical but also intuitive and adapted to real-world needs.

The use of user stories and personas; which are key to Agile planning, further guaranteed that the system was aligned with user expectations. Each requirement was likely established with a specific end-user situation in mind, ensuring clarity and purpose for each feature. This helped the design team prioritize high-value functions such as secure login, real-time SMS confirmations, and various location input methods, which directly impact user satisfaction and service efficiency.

Moreover, Agile's modular and adaptive nature complemented the technical requirements of the project. The system design exhibits a modular architecture, which facilitates maintainability and future upgrades; a fundamental Agile benefit. As the business scales or customer needs evolve, additional features like enhanced rating filters, multilingual support, or advanced analytics can be deployed without redesigning the entire system. This adaptability is vital for a firm like Fresh Grocers, which works in a competitive and continually expanding sector.

Agile also emphasized early and regular testing, which is crucial in assuring the robustness of a platform that handles sensitive data and real-time interactions. By regularly testing each module such as registration, ordering, or location tracking, developers could find bugs, performance issues, or security concerns at early stages. This proactive testing culture ultimately leads to a more reliable and secure final product.

Despite these characteristics, there are situations where Agile's implementation could have been enhanced or more clearly documented. For instance, while the system architecture supports many Agile concepts, it would have been great to explicitly explain how sprint planning, user feedback, and backlog grooming were managed. Additionally, integrating automated technologies for continuous integration and deployment (CI/CD), a typical Agile best practice was addressed in the system requirements but not clearly shown in the design artifacts. Ensuring that these principles are integrated into the Agile workflow is critical for maintaining the pace and quality of iterative releases.

In conclusion, Agile was a good solution for the Fresh Grocers system design project. Its iterative nature, focus on user feedback, and modular architecture allowed the team to design a scalable, user-friendly, and robust platform.Agile not only supported the technical growth of the system but also guaranteed that the increasing needs of a broad user base were constantly satisfied. With minimal modifications in documentation and installation of automation technologies, the Agile methodology might continue to serve as a strong basis for future phases of the project.

4.5 Assessing the Effectiveness of the System Design

Assessing the efficacy of the Fresh Grocers system design needs a complete review of several key components, including the methodology applied, the design process followed, and the structural and visual features that contribute to the overall system. Each of these characteristics plays a key role in deciding how successfully the system accomplishes its intended objectives, facilitates user interaction, and assures operational efficiency.

The chosen approach for the project was Agile, which proved to be highly suitable for a system of this sort. Agile stresses iterative development, ongoing stakeholder communication, and the flexibility to adapt to changing requirements. For a platform like Fresh Grocers; serving a wide spectrum of users such as customers, delivery personnel, and customer care representatives. Agile helped the split of the project into manageable chunks. Each component, such as registration, order placement, delivery management, and rating systems, could be designed, tested, and refined in short sprints. This allowed the development team to collect feedback from real users, make timely modifications, and ensure the final product was user-centered and responsive to evolving needs.

The design approach followed a logical and structured course. Beginning with the discovery of user needs, the team built clear wireframes and prototypes for each main function. These featured user registration, login, and order placing by consumers and CSRs, delivery agent dashboards, and rating interfaces. The design process paid special attention to the flow of actions within the system, ensuring that each function was supported by intuitive layouts and logical transitions. By combining both low-fidelity wireframes and high-fidelity prototypes, the team ensured that both conceptual and visual elements were examined and validated.

The user interface design is one of the system's most remarkable characteristics. It is straightforward, intuitive, and adapted to the demands of varied user groups. Customers may navigate quickly through the grocery selection and ordering process, while CSRs have access to specific forms that streamline order placement on behalf of customers. The interface for delivery agents is equally user-friendly, allowing them to update their status and examine allocated orders conveniently. The visual hierarchy is maintained across the platform, with clean layouts and logical button placements that reduce cognitive burden and encourage use. These traits are particularly significant for non-tech-savvy users and reflect that the design team placed a strong premium on accessibility and ease of use.

The system architecture exhibits a well-structured and scalable design. It divides the presentation layer, business logic, and data access layers, facilitating modular development and easy maintenance. The architecture is designed to manage high levels of user interaction and data processing, which is required for a system expected to run in real-time with simultaneous users placing orders, updating locations, and sending notifications. The modular design also enables for easy integration of new features or third-party APIs, such as SMS gateways or email services, which is vital for future-proofing the platform.

The database architecture is successful in expressing the relationships and data flows within the system. Core entities such as users, orders, ratings, and locations are properly defined, and their relationships are rationally constructed. This enables reliable data retrieval and storage, which is critical for preserving service integrity. The design covers crucial functions such as tracking user activity, managing delivery statuses, and saving feedback. However, the inclusion of security-specific fields such as encrypted password storage and access roles, could be more explicitly expressed to completely accord with best practices in data protection.

Program design, including flowcharts and pseudocode, indicates a solid understanding of the rationale behind each feature. Processes like as user registration, customer ordering, CSR engagement, and delivery management are well-sequenced and easy to follow. The use of flowcharts helps illustrate the decision-making processes and user interactions, while pseudocode describes the steps necessary for developers to implement these features. However, there is limited mention of error management, data validation, and security procedures in the software design. Including these aspects would have strengthened the system's robustness and indicated a more holistic development plan.

In conclusion, the system design for Fresh Grocers is mainly effective and well-aligned with the project's aims. The implementation of the Agile methodology allowed flexibility and user involvement, while the design process ensured that all essential user interactions were properly planned and tested. The user interface is intuitive and accessible, the system architecture is scalable and maintainable, the database is logically structured, and the program design provides a solid foundation for implementation. While there is space for improvement in defining specific backend procedures and security issues, the overall design presents a powerful, practical, and user-focused solution.